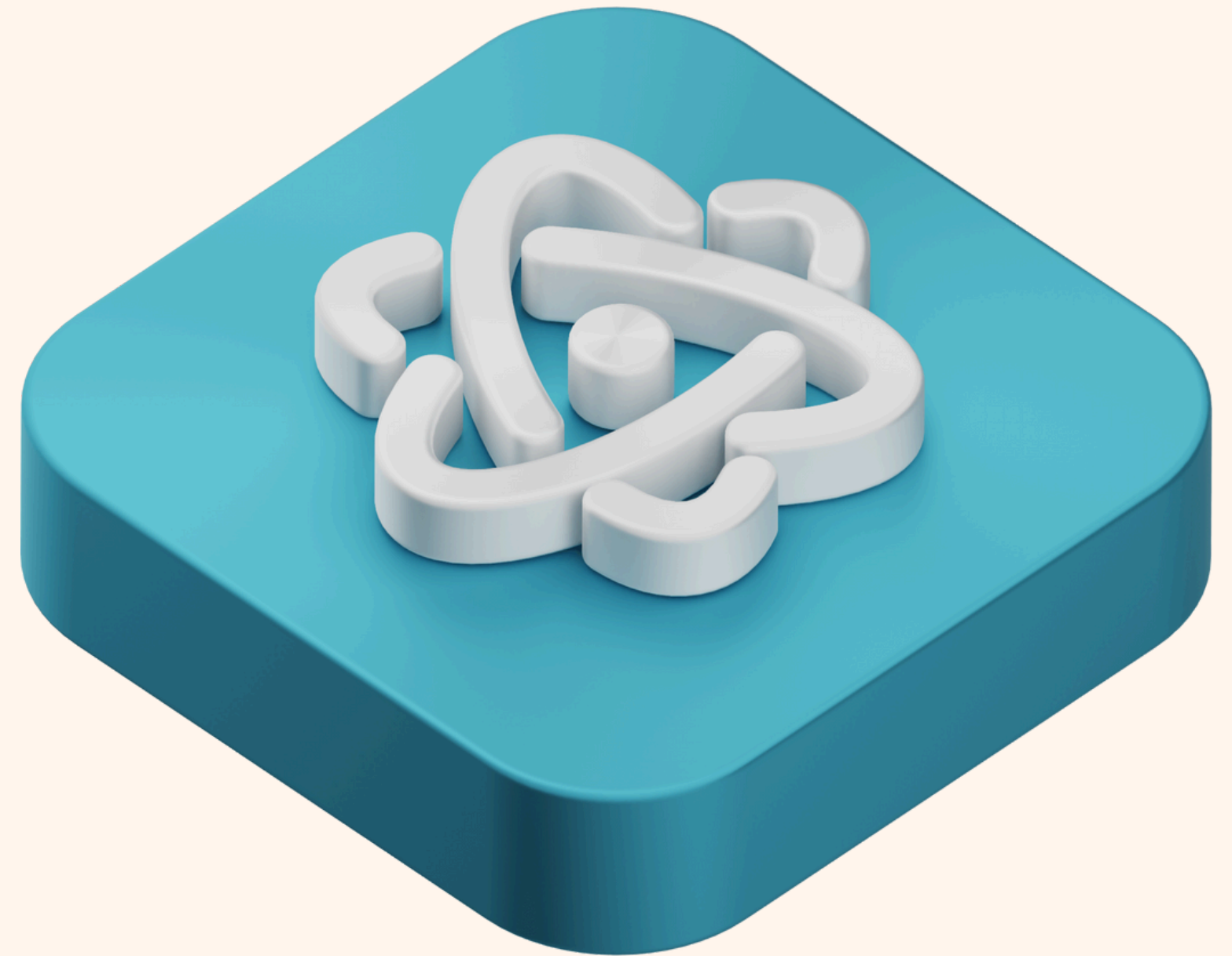


FRONTEND DEEP DIVE

(CONTINUED)

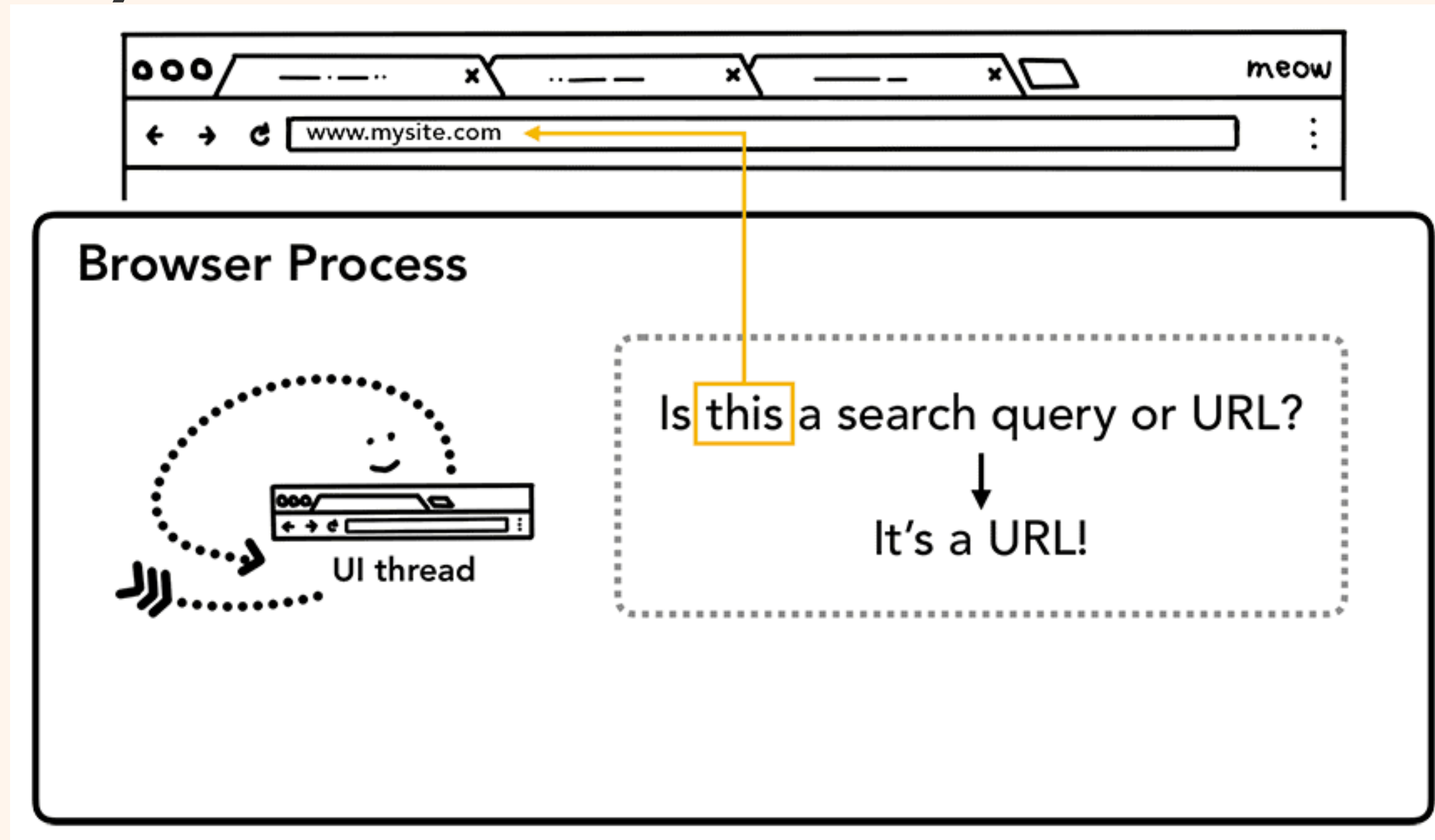


CRITICAL RENDERING PATH

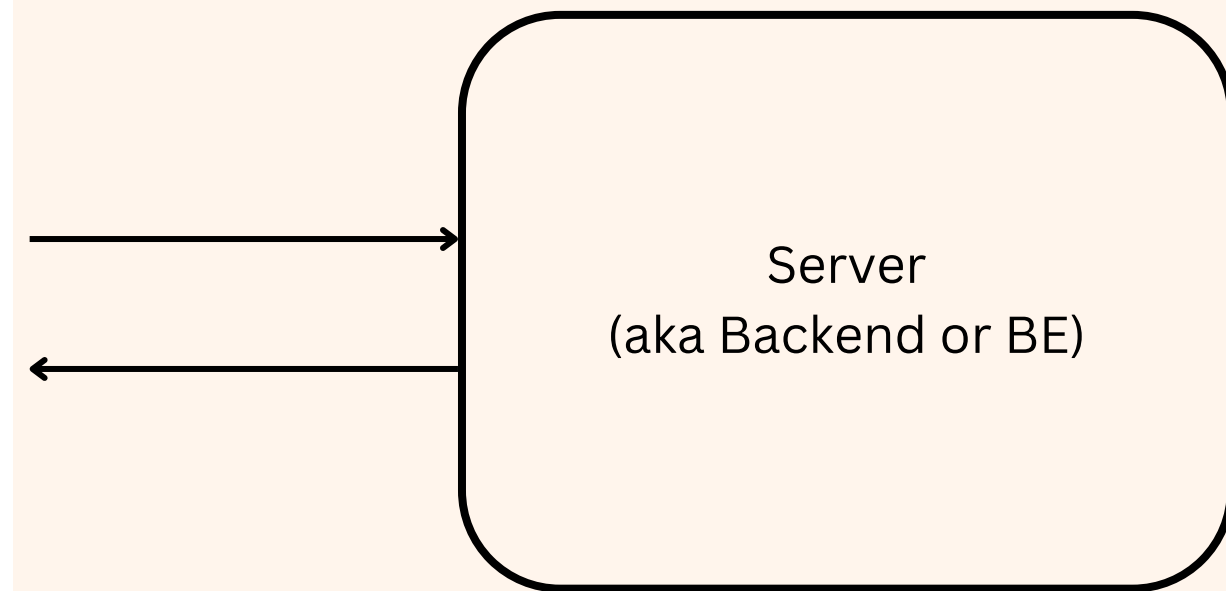
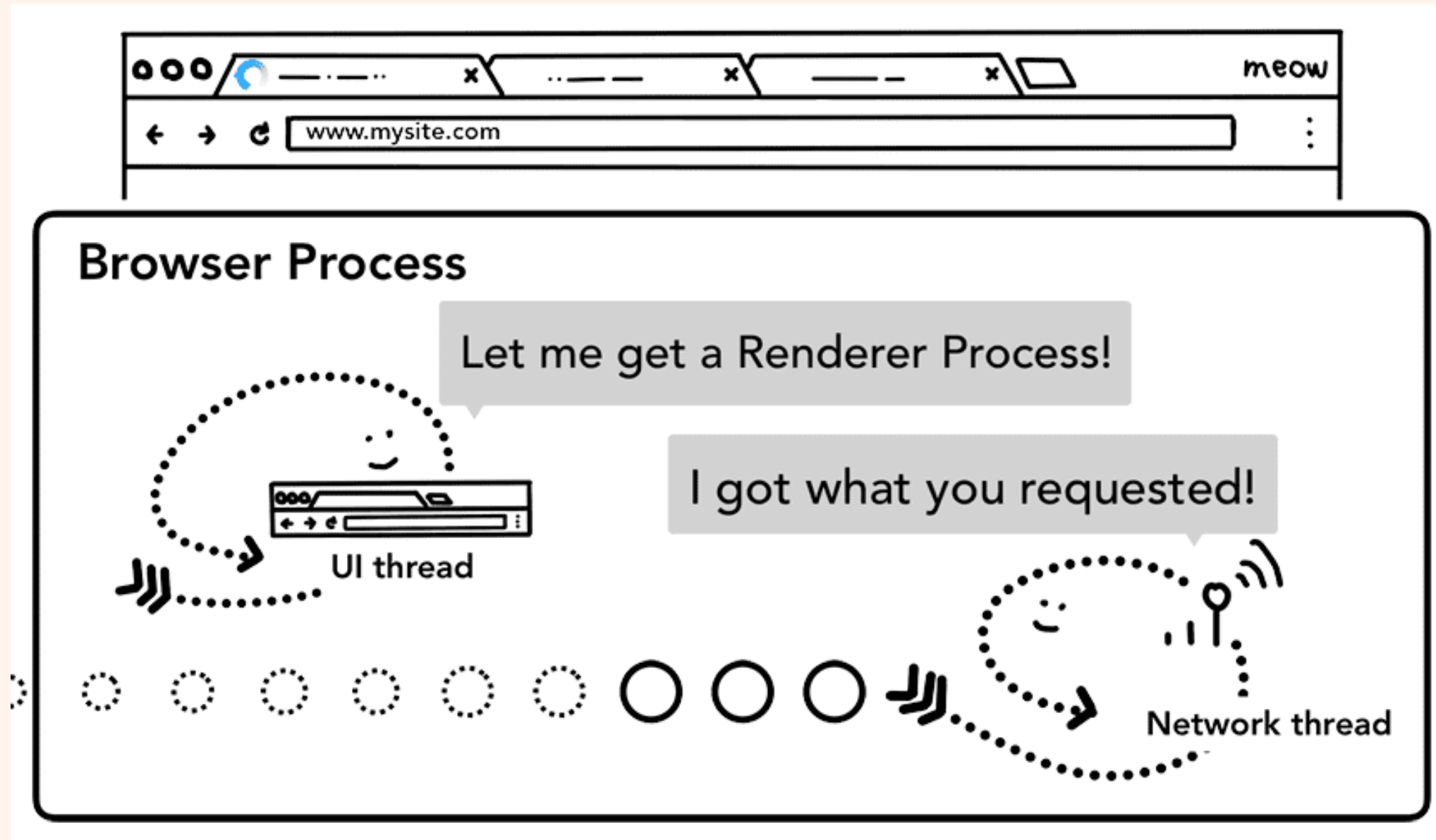
1. Get the initial HTML 
2. Parse the HTML into the DOM
3. Parse CSS into the CSSOM
4. Create render tree
5. Determine where render tree nodes should appear on the page (layout)
6. Paint nodes onto screen (rasterize)

1. GETTING THE INITIAL HTML

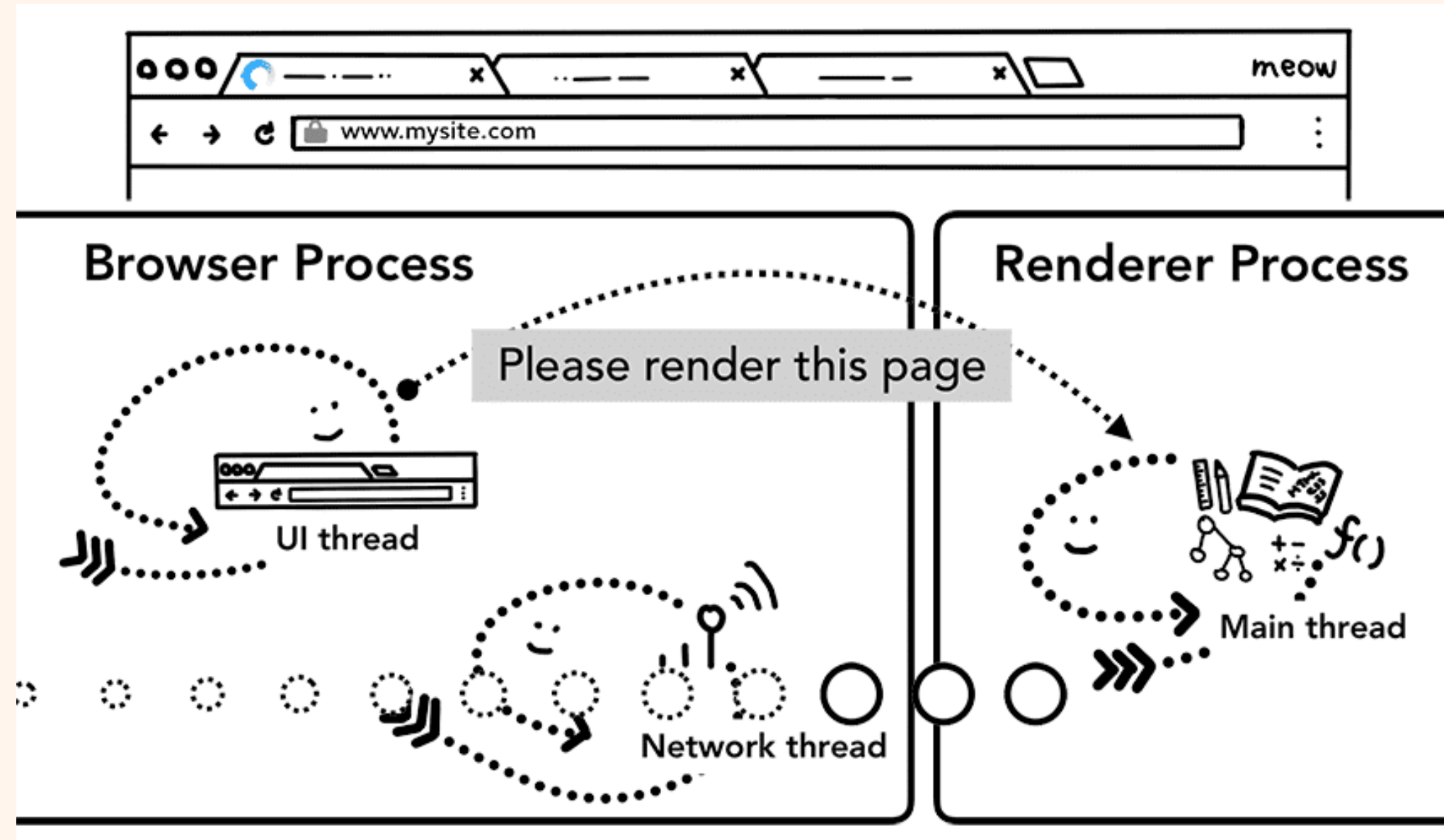
In your browser, navigate to a website of your choice



1. GETTING THE INITIAL HTML



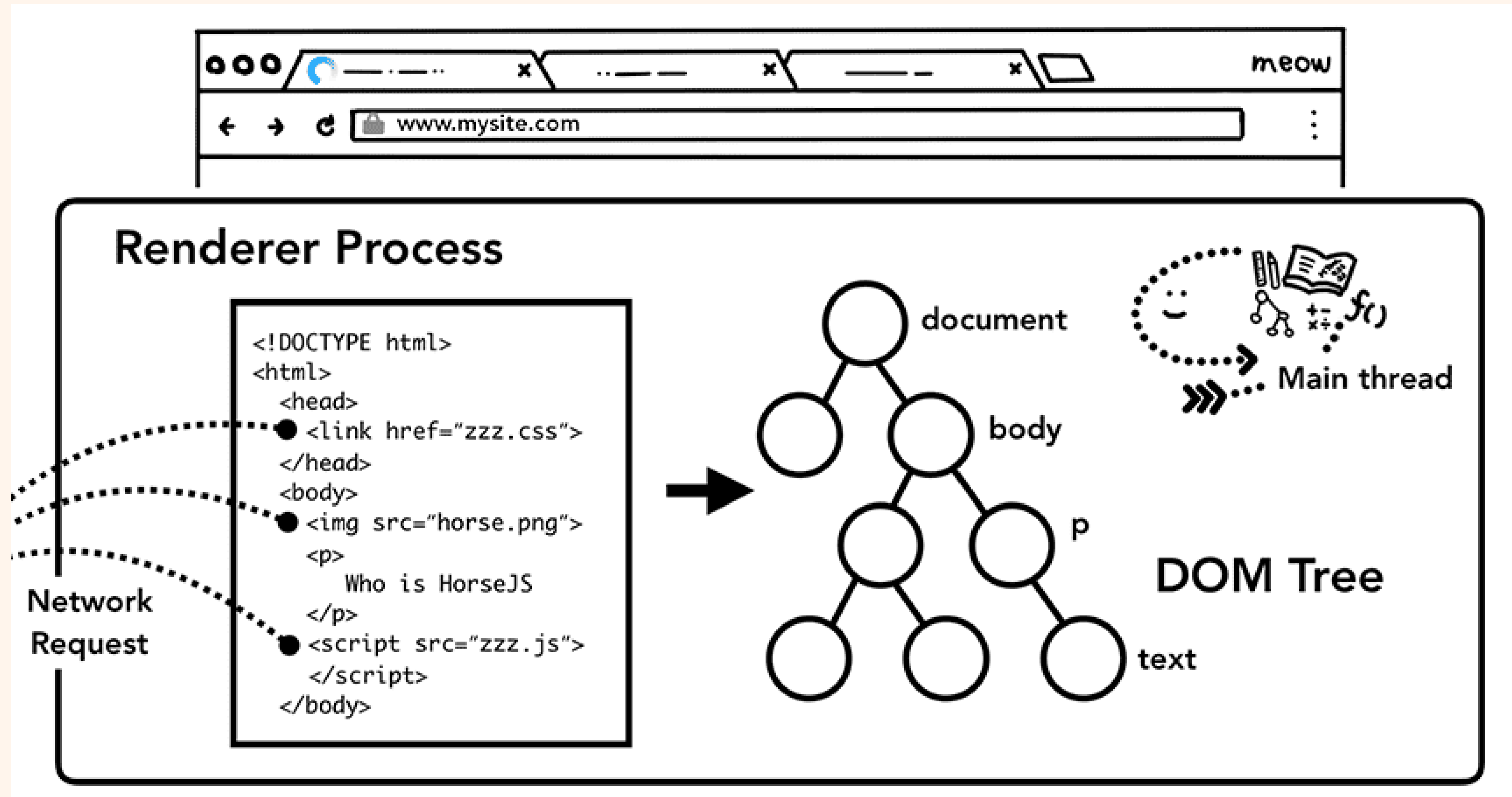
1. GETTING THE INITIAL HTML



CRITICAL RENDERING PATH

1. Get the initial HTML
2. Parse the HTML into the DOM 
3. Parse CSS into the CSSOM
4. Create render tree
5. Determine where render tree nodes should appear on the page (layout)
6. Paint nodes onto screen (rasterize)

2. PARSE THE HTML INTO THE DOM

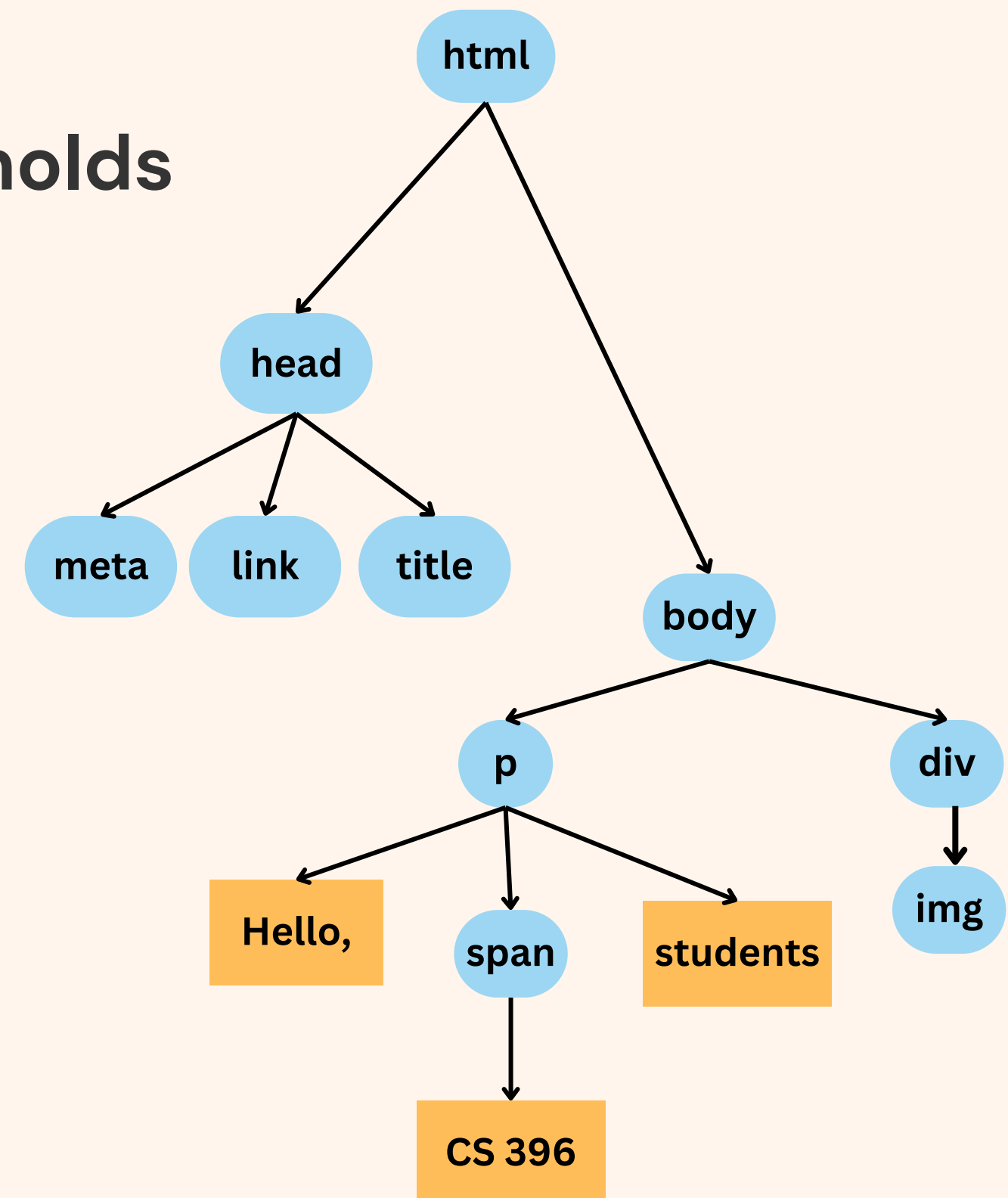


2. PARSE THE HTML INTO THE DOM

DOM = Document Object Model

- Browser's tree data structure that holds what goes into a page

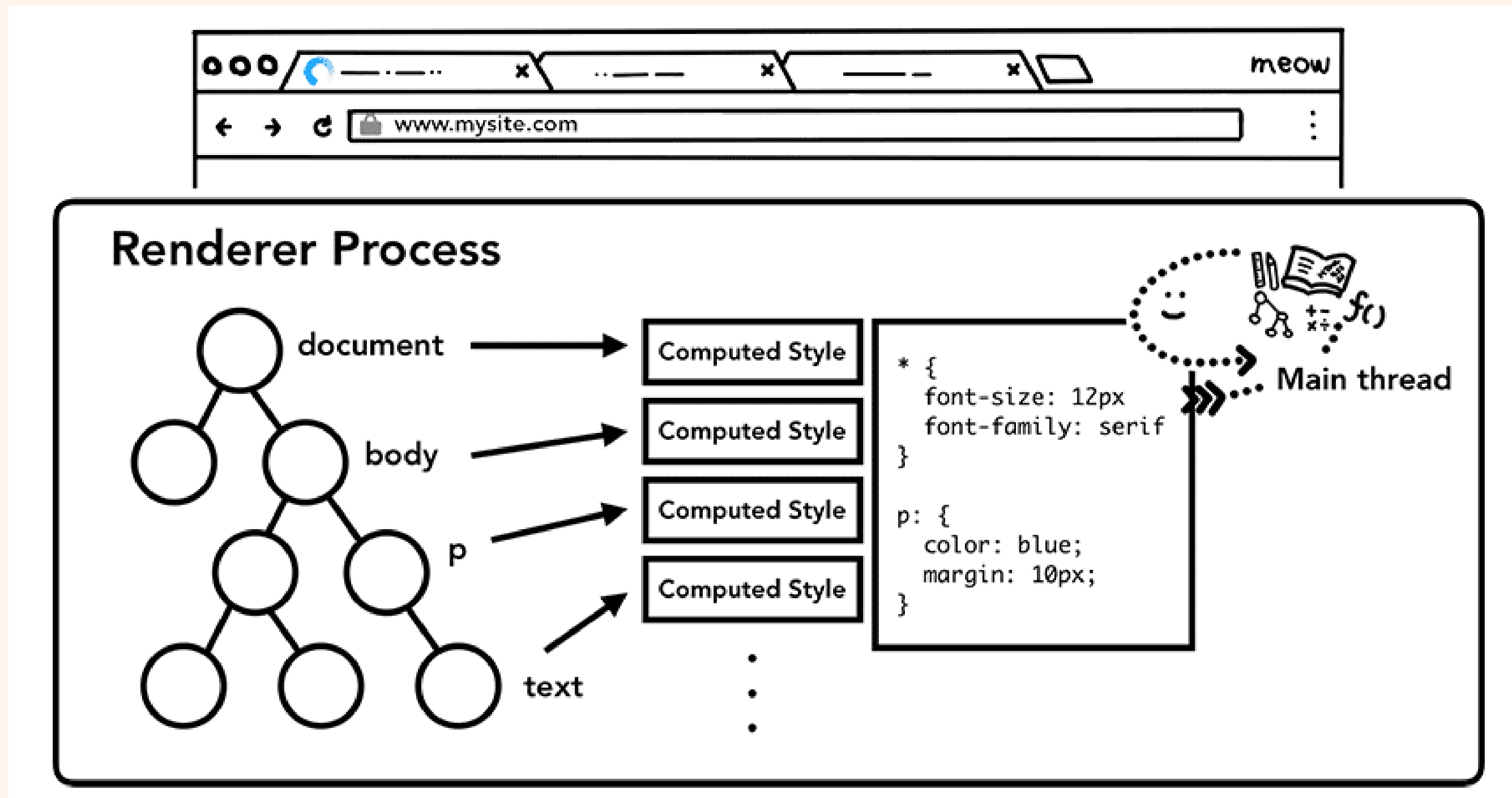
```
<!DOCTYPE html>
<html>
  <head>
    <meta name="viewport" content="width=device-width,initial-scale=1" />
    <link href="style.css" rel="stylesheet" />
    <title>DOM + CSSOM Trees</title>
  </head>
  <body>
    <p>Hello <span>CS 396</span> students</p>
    <div></div>
  </body>
</html>
```



CRITICAL RENDERING PATH

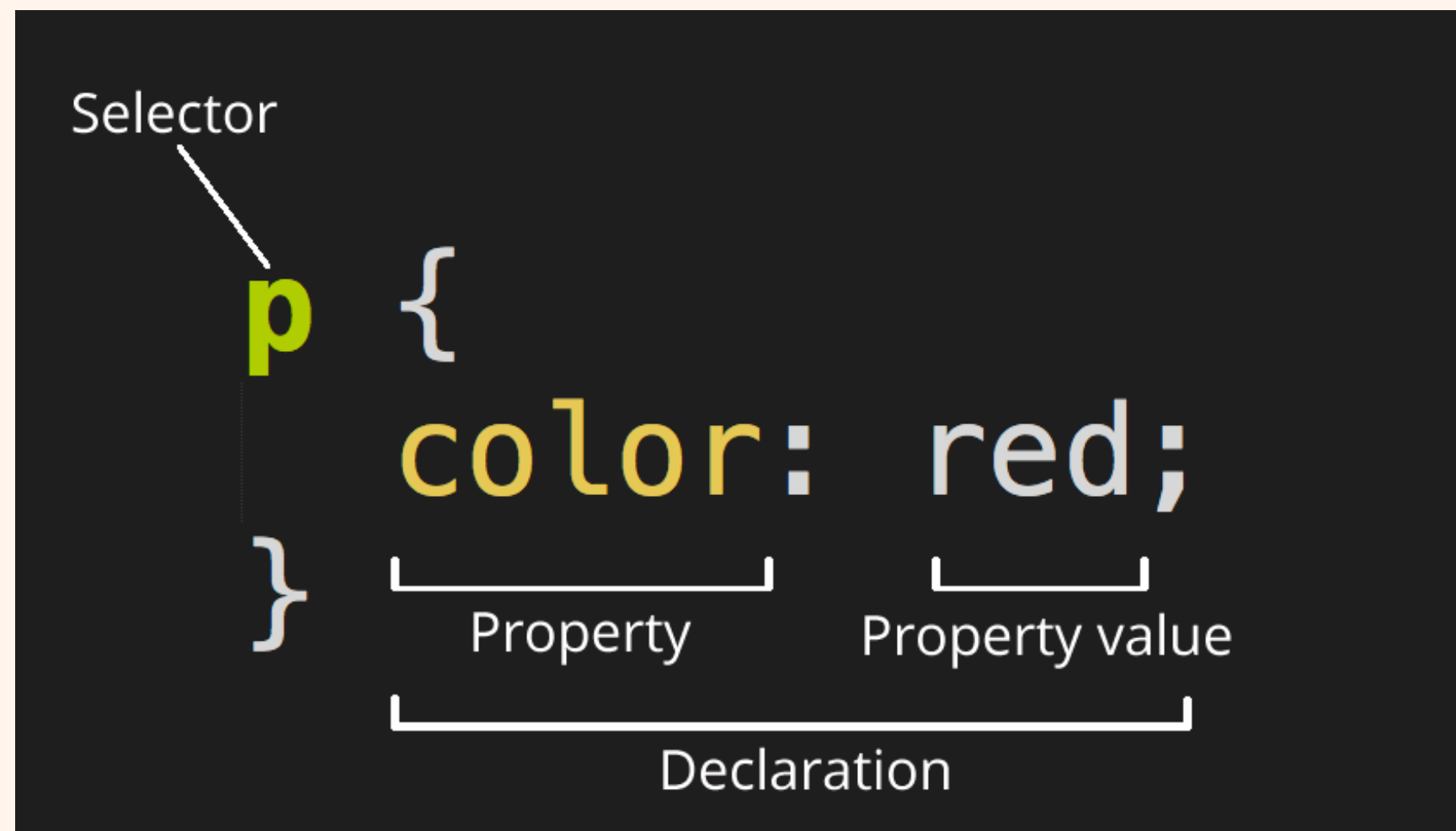
1. Get the initial HTML
2. Parse the HTML into the DOM
3. Parse CSS into the CSSOM 
4. Create render tree
5. Determine where render tree nodes should appear on the page (layout)
6. Paint nodes onto screen (rasterize)

3. PARSE CSS INTO THE CSSOM



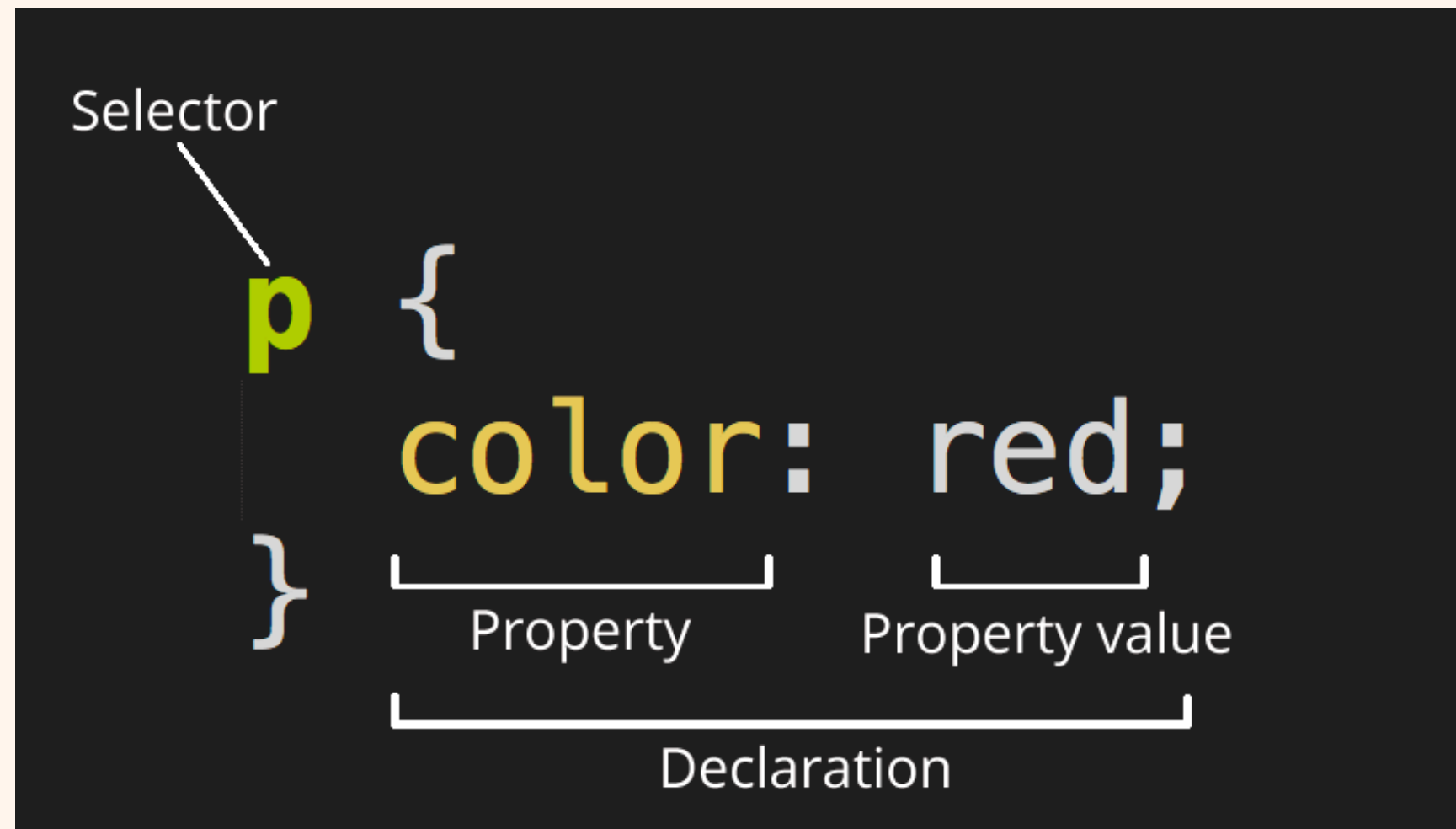
3. PARSE CSS INTO THE CSSOM

```
body {  
  font-size: 16px;  
}  
p {  
  font-weight: bold;  
}  
span {  
  color: red;  
}  
p span {  
  display: none;  
}  
img {  
  float: right;  
}
```



3. PARSE CSS INTO THE CSSOM

```
body {  
  font-size: 16px;  
}  
p {  
  font-weight: bold;  
}  
span {  
  color: red;  
}  
p span {  
  display: none;  
}  
img {  
  float: right;  
}
```

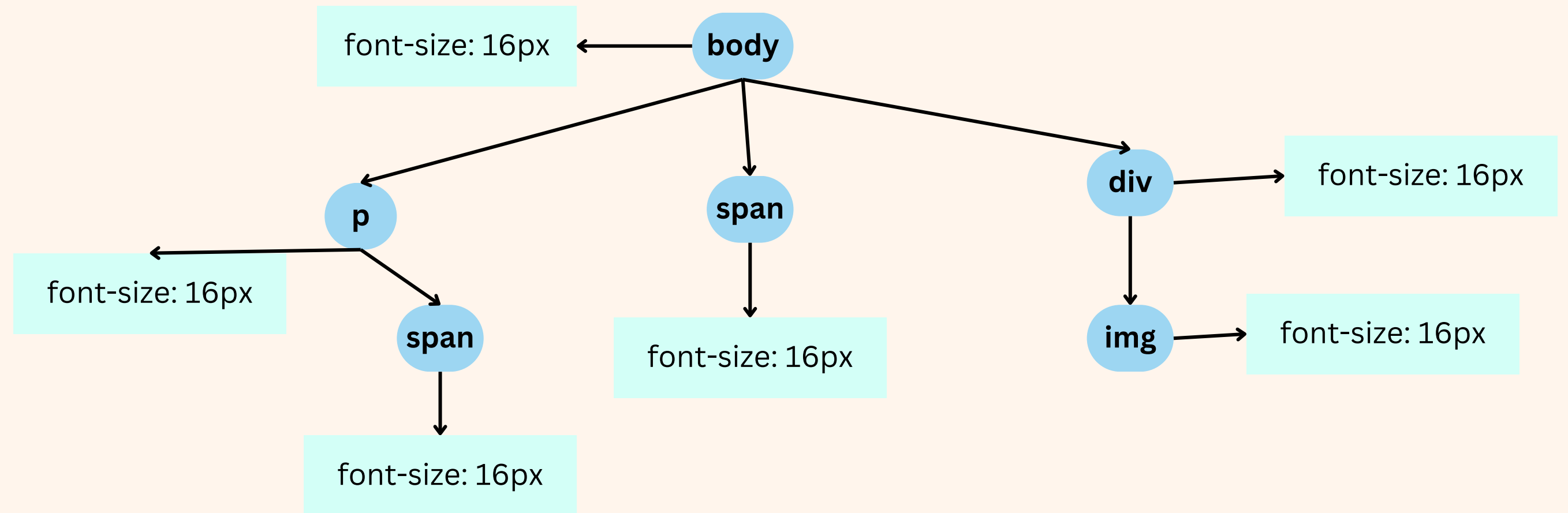


Your turn:

- Choose an element in your index.html to add styling

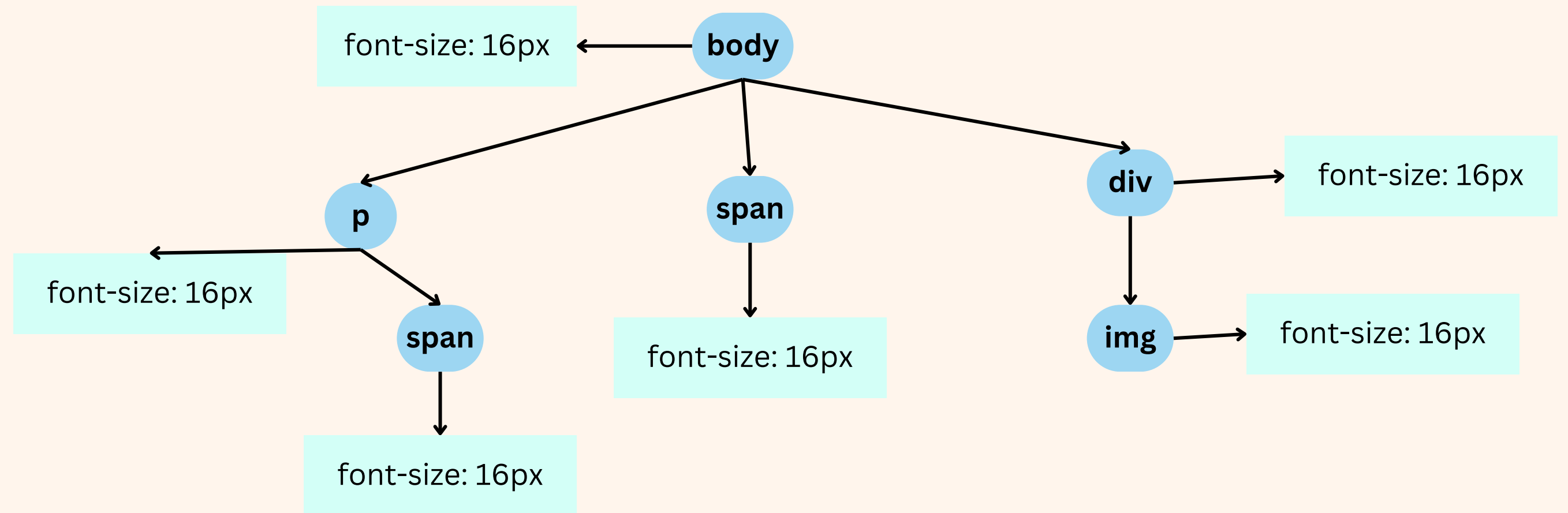
3. PARSE CSS INTO THE CSSOM

```
body {  
  font-size: 16px;  
}  
p {  
  font-weight: bold;  
}  
span {  
  color: red;  
}  
p span {  
  display: none;  
}  
img {  
  float: right;  
}
```



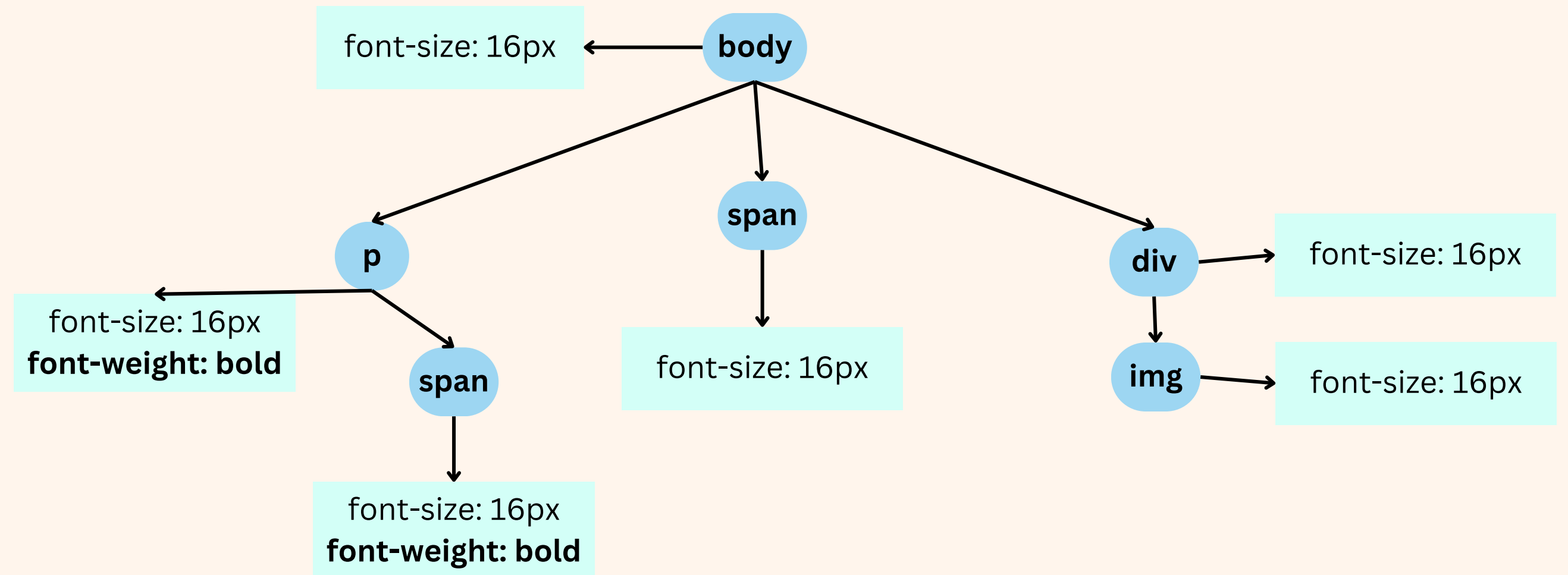
3. PARSE CSS INTO THE CSSOM

```
body {  
  font-size: 16px;  
}  
p {  
  font-weight: bold;  
}  
span {  
  color: red;  
}  
p span {  
  display: none;  
}  
img {  
  float: right;  
}
```



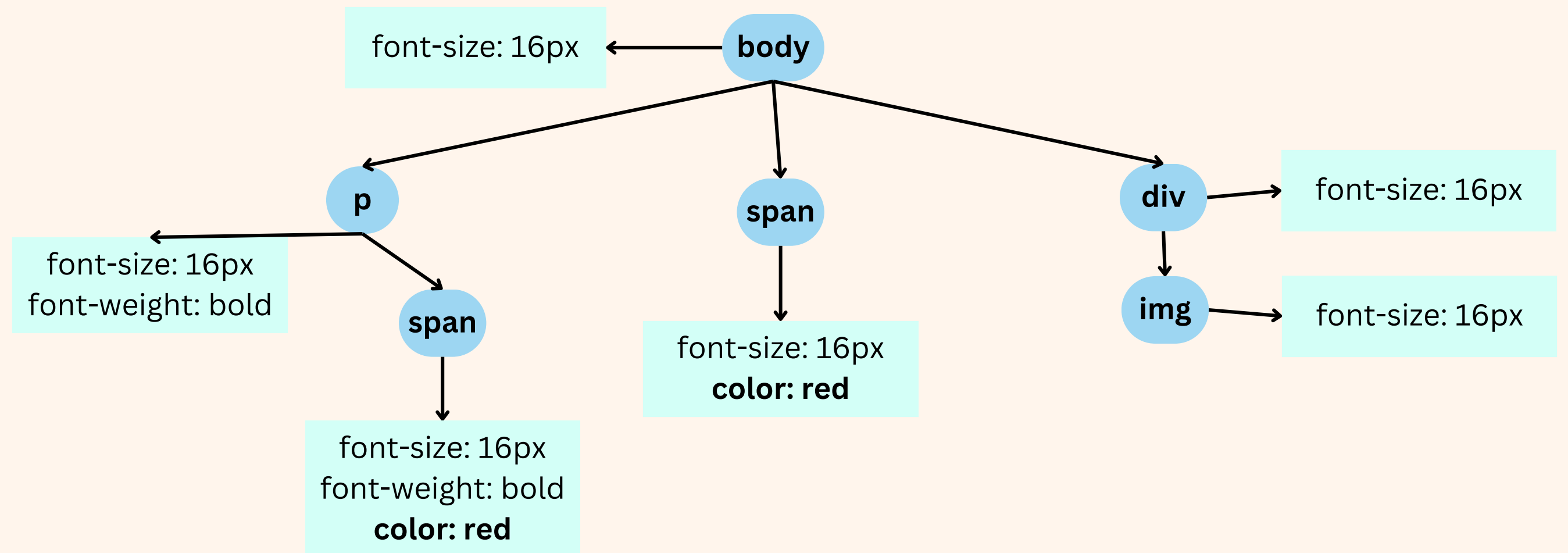
3. PARSE CSS INTO THE CSSOM

```
body {  
  font-size: 16px;  
}  
p {  
  font-weight: bold;  
}  
span {  
  color: red;  
}  
p span {  
  display: none;  
}  
img {  
  float: right;  
}
```



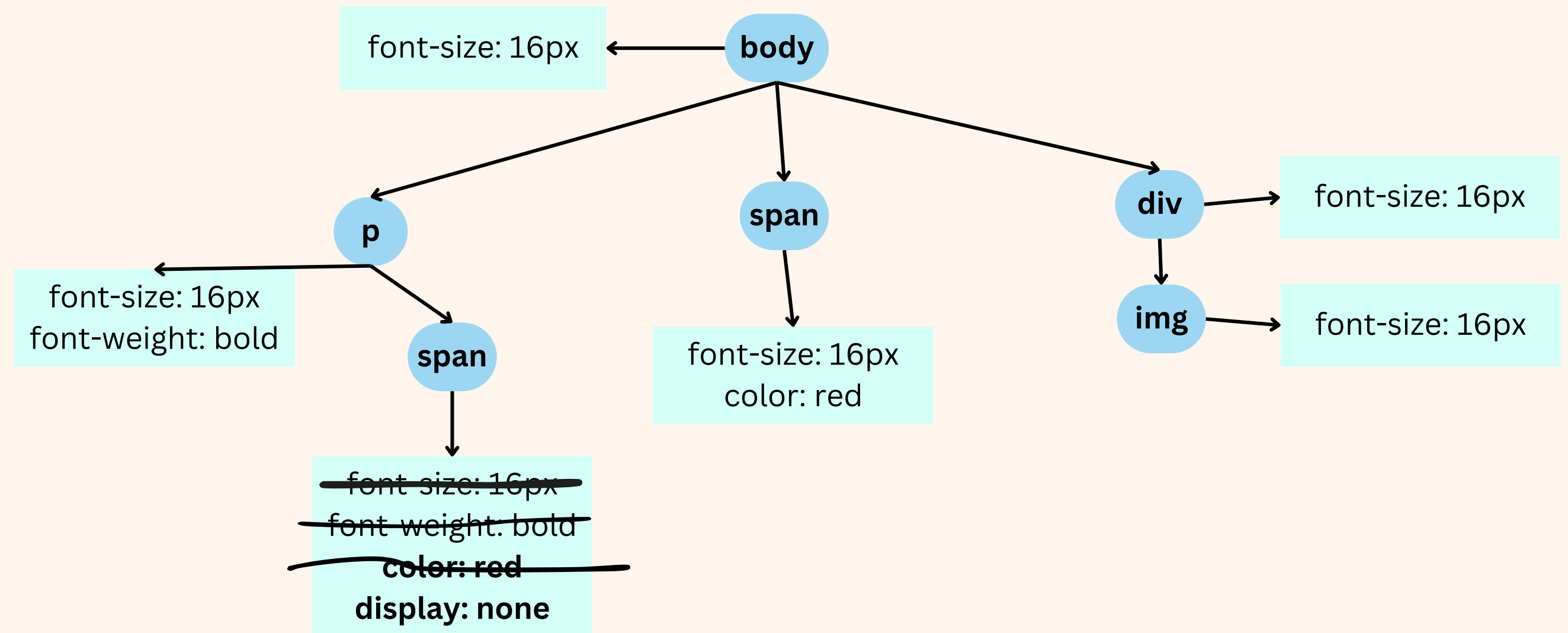
3. PARSE CSS INTO THE CSSOM

```
body {  
  font-size: 16px;  
}  
p {  
  font-weight: bold;  
}  
span {  
  color: red;  
}  
p span {  
  display: none;  
}  
img {  
  float: right;  
}
```



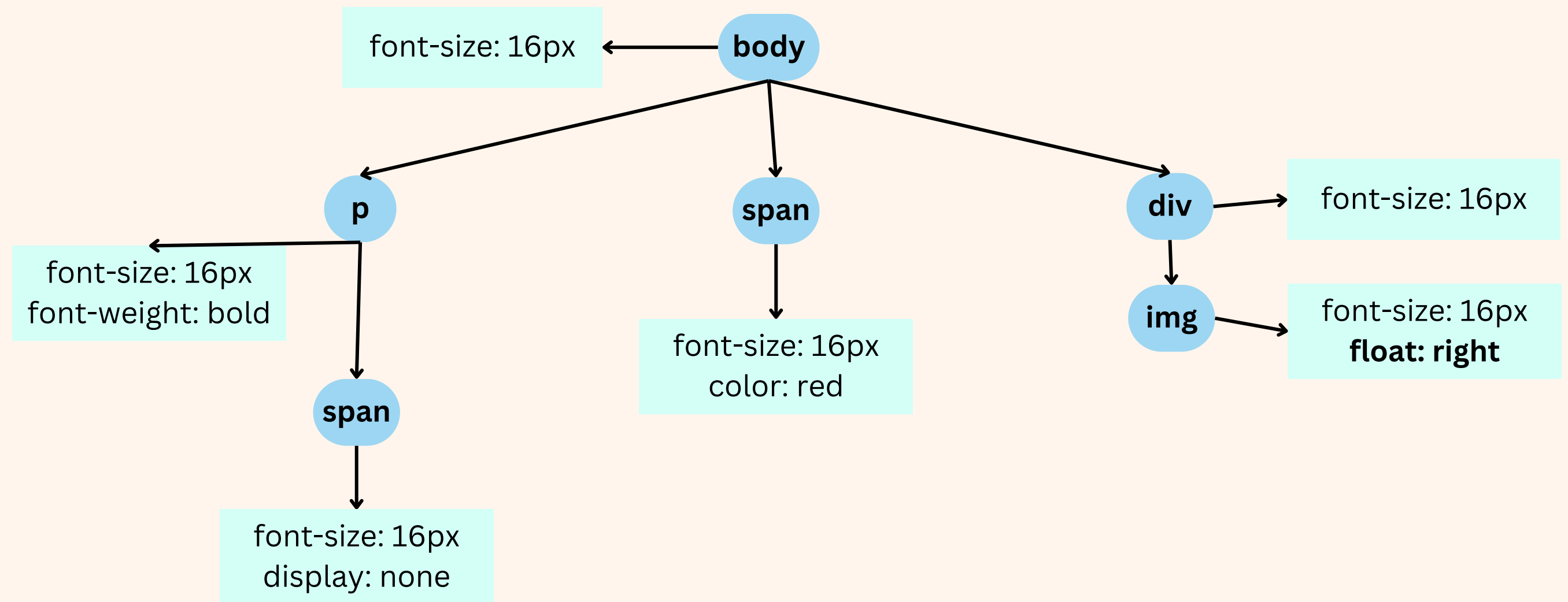
3. PARSE CSS INTO THE CSSOM

```
body {  
  font-size: 16px;  
}  
p {  
  font-weight: bold;  
}  
span {  
  color: red;  
}  
p span {  
  display: none;  
}  
img {  
  float: right;  
}
```



3. PARSE CSS INTO THE CSSOM

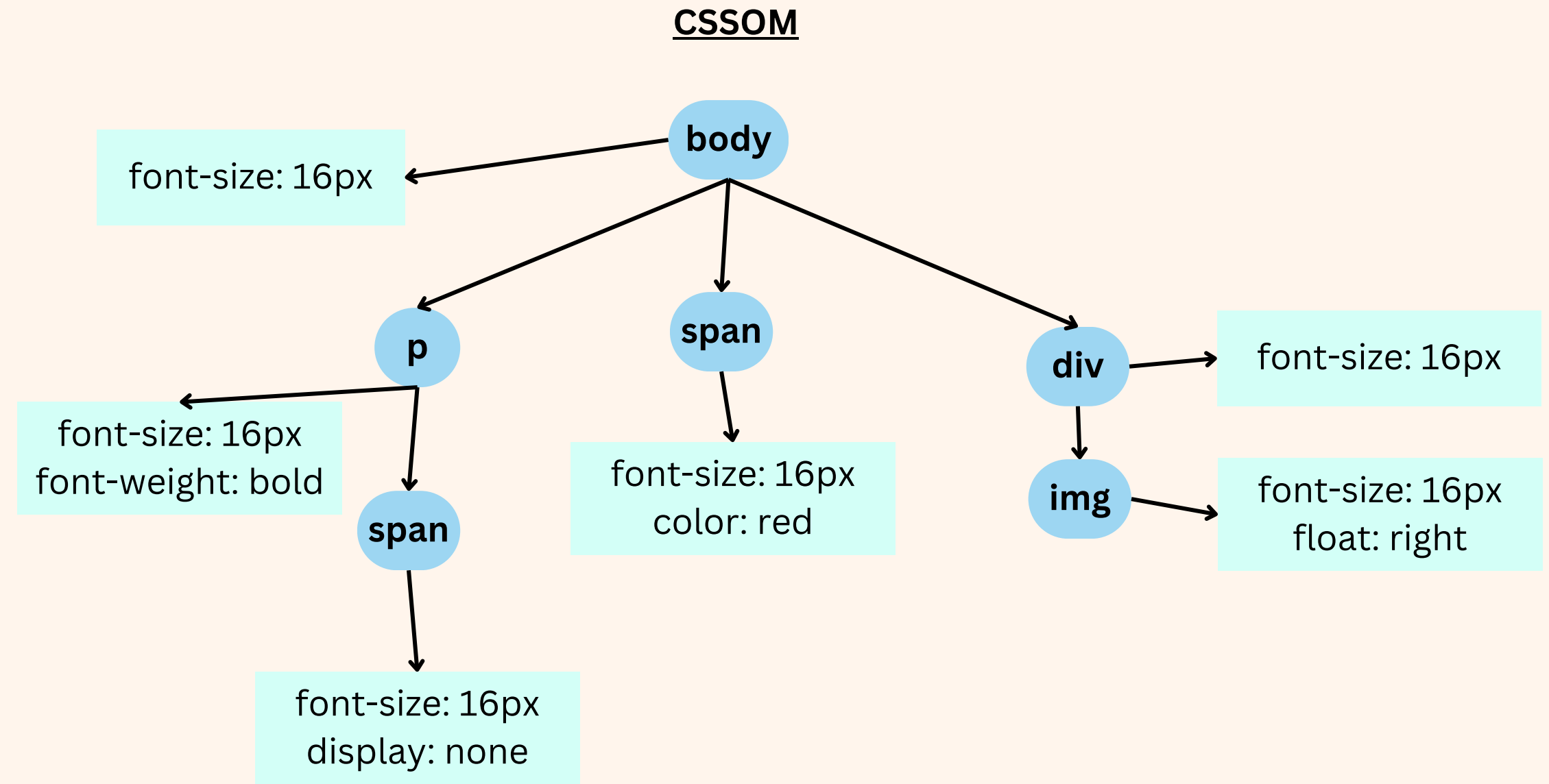
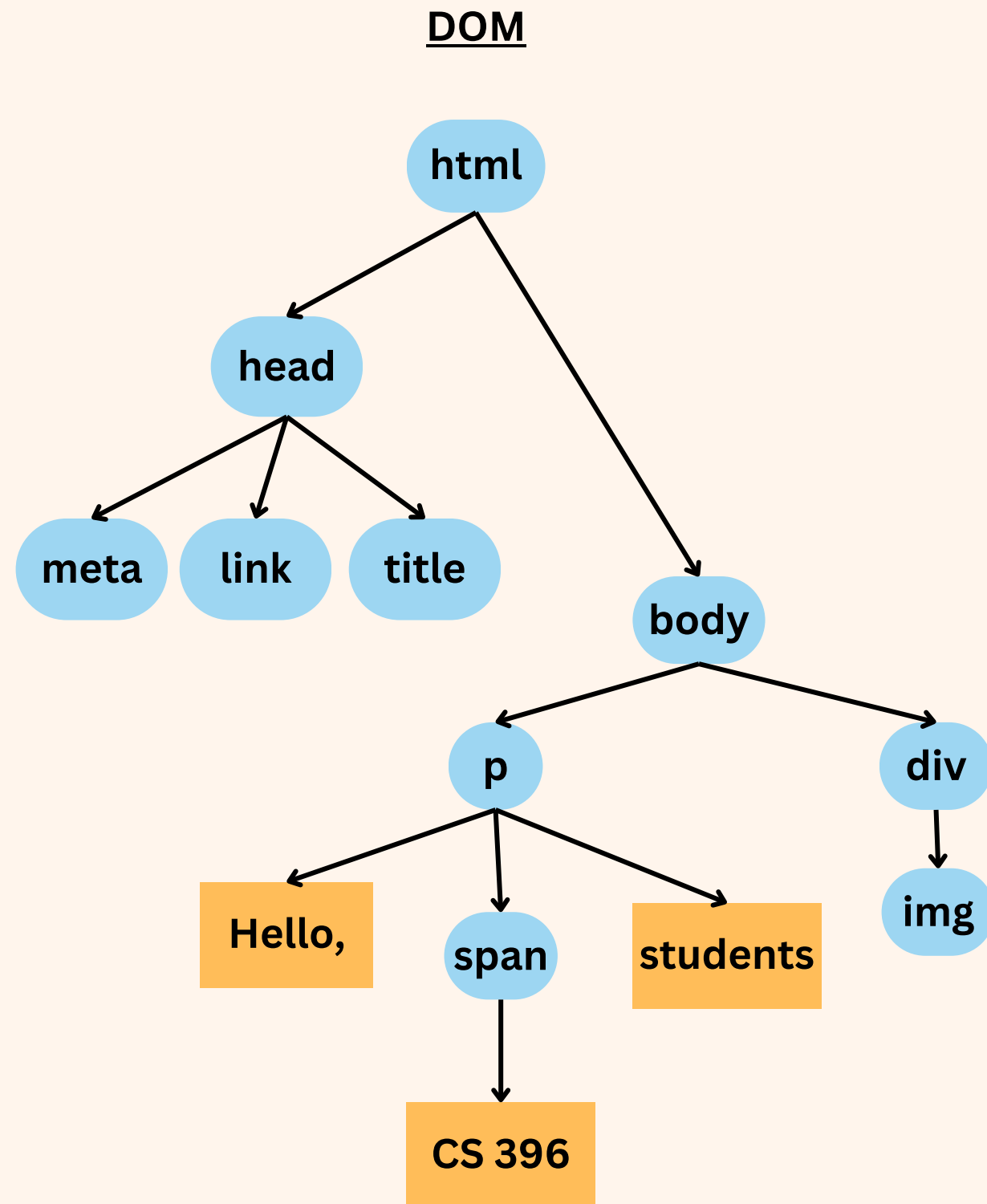
```
body {  
  font-size: 16px;  
}  
p {  
  font-weight: bold;  
}  
span {  
  color: red;  
}  
p span {  
  display: none;  
}  
img {  
  float: right;  
}
```



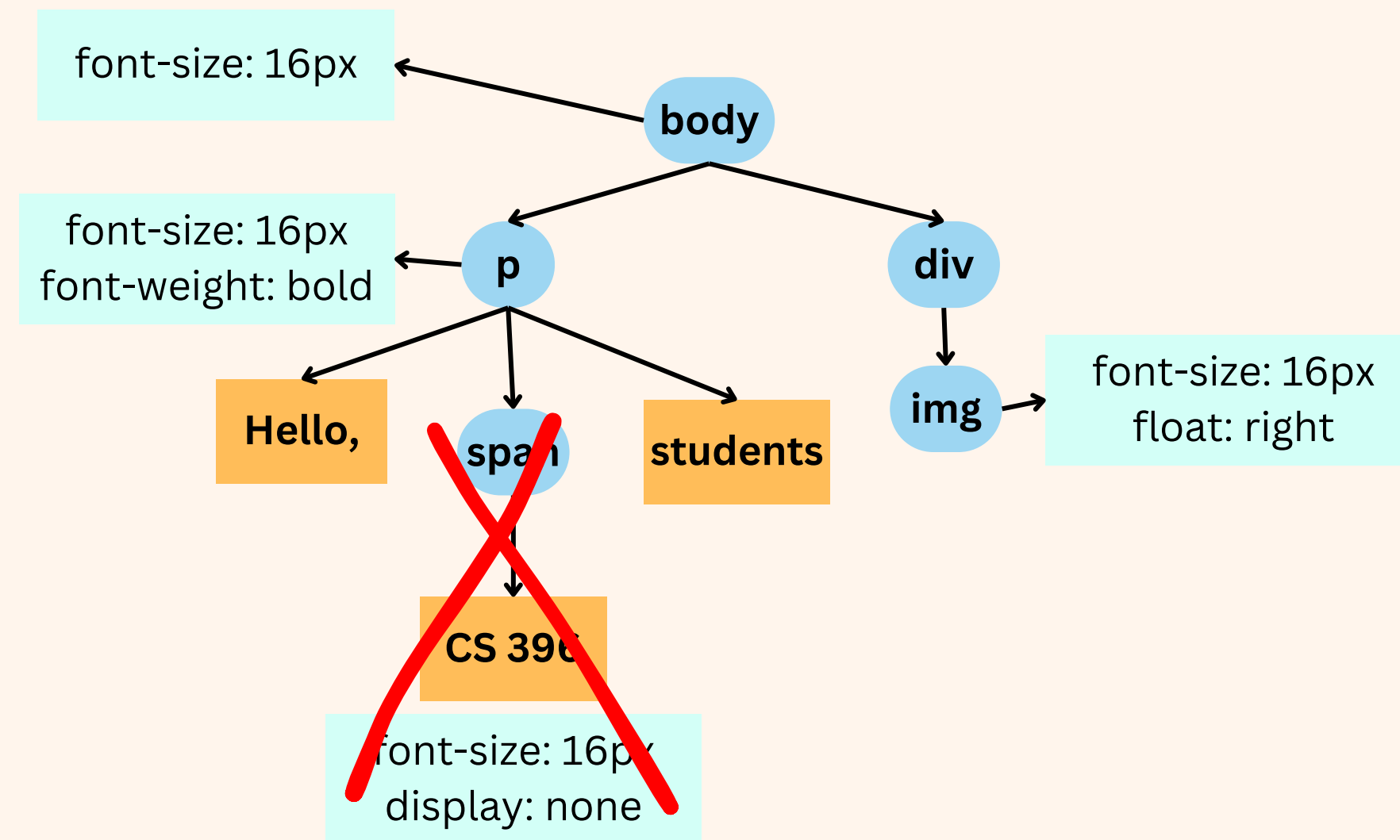
CRITICAL RENDERING PATH

1. Get the initial HTML
2. Parse the HTML into the DOM
3. Parse CSS into the CSSOM
4. Create render tree 
5. Determine where render tree nodes should appear on the page (layout)
6. Paint nodes onto screen (rasterize)

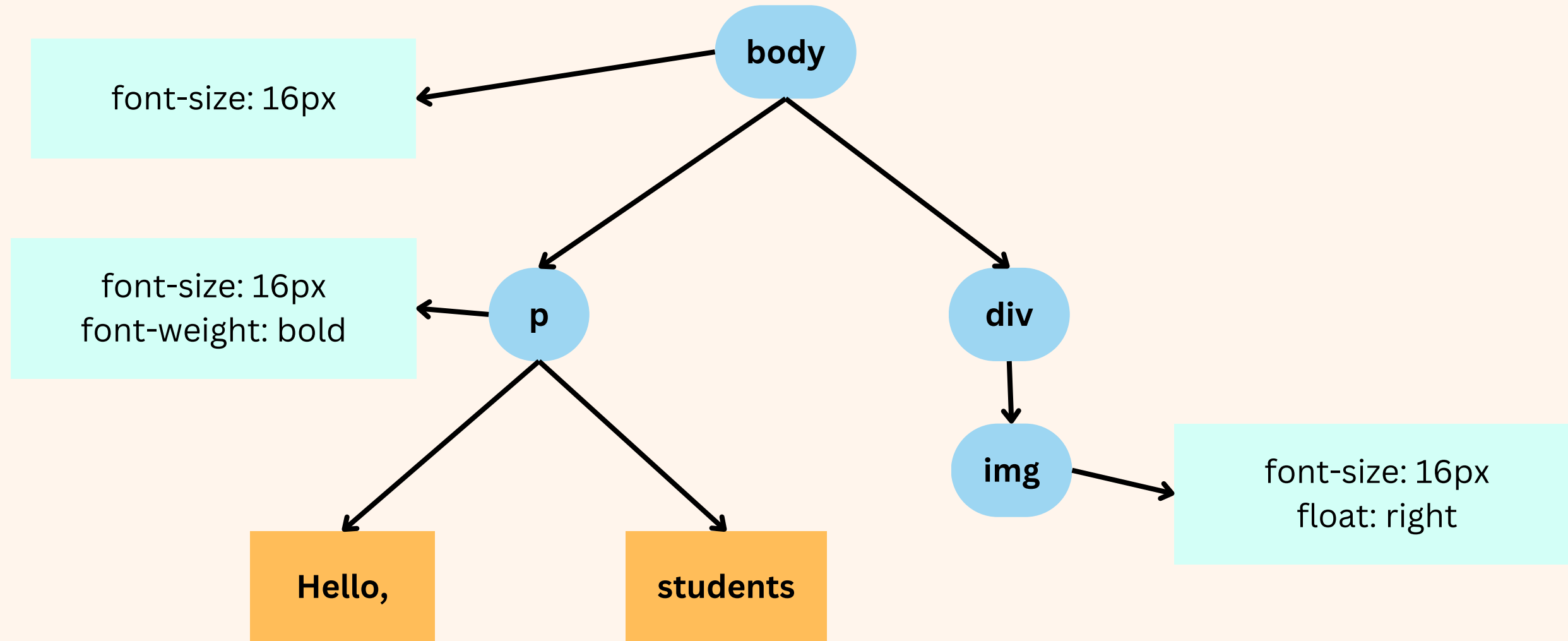
4. CREATE RENDER TREE



4. CREATE RENDER TREE

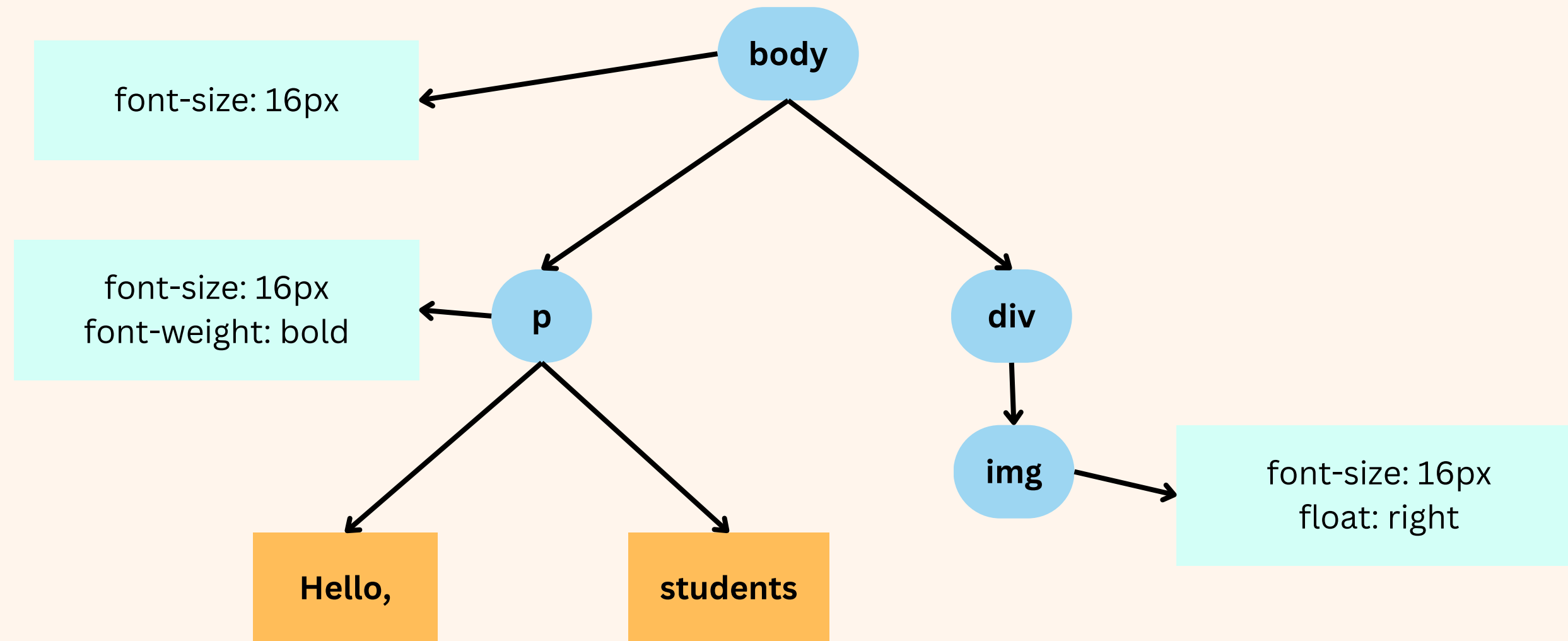


4. CREATE RENDER TREE



Resultant Render Tree

4. CREATE RENDER TREE



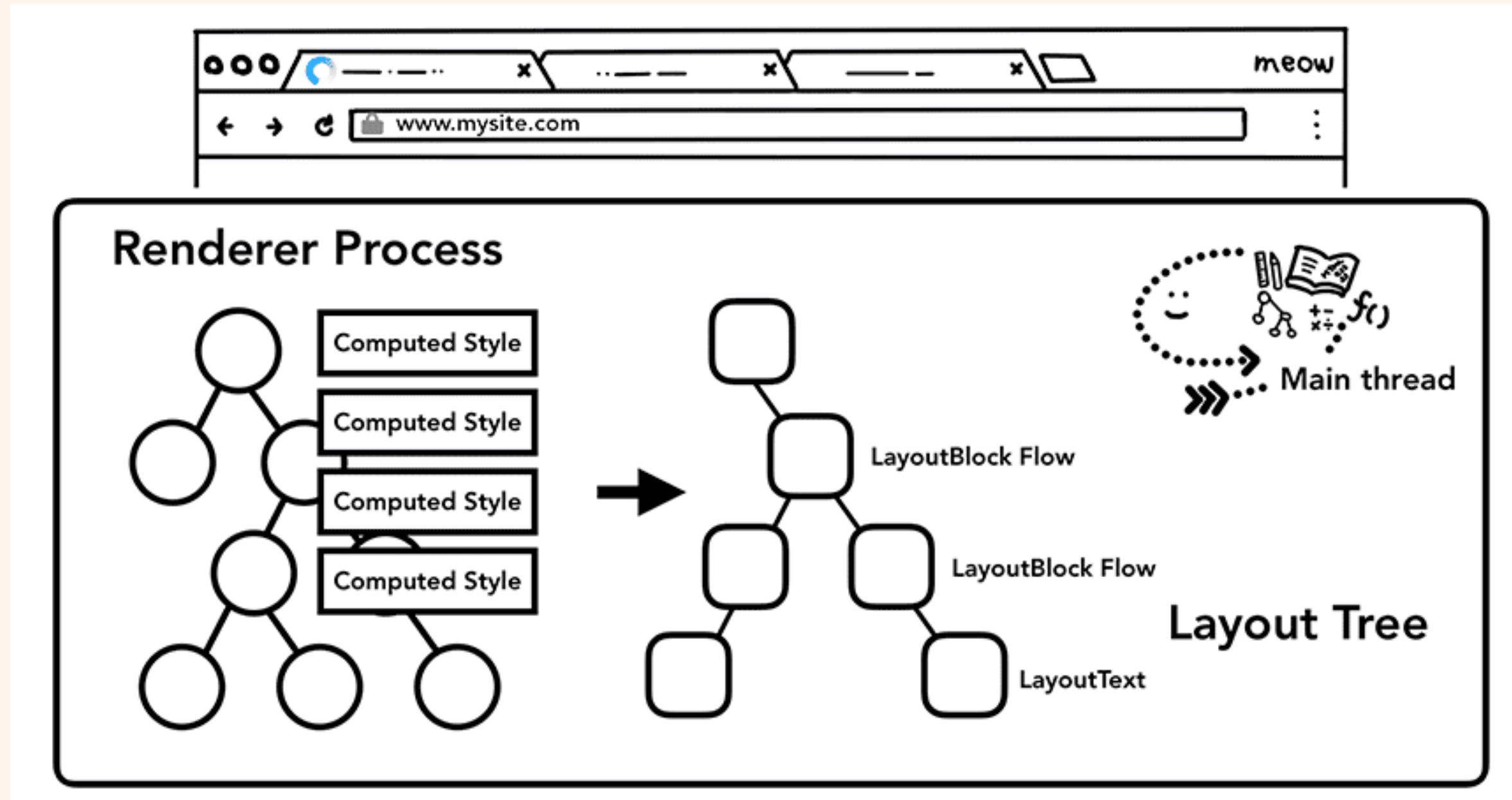
Resultant Render Tree

CRITICAL RENDERING PATH

1. Get the initial HTML
2. Parse the HTML into the DOM
3. Parse CSS into the CSSOM
4. Create render tree
5. Determine where render tree nodes should appear on the page (layout)
6. Paint nodes onto screen (rasterize)

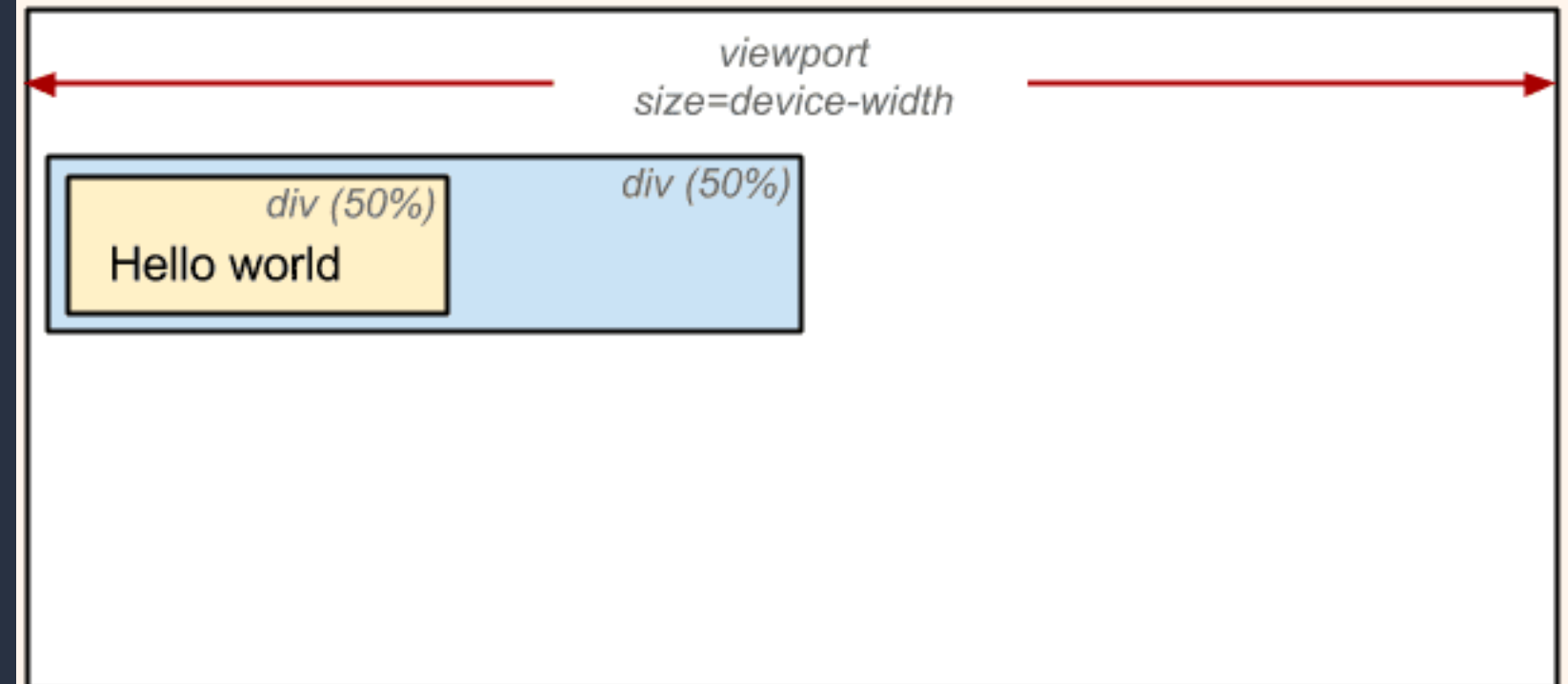


5. LAYOUT



5. LAYOUT

```
<!DOCTYPE html>
<html>
  <head>
    <meta name="viewport" content="width=device-width,initial-scale=1" />
    <title>Critical Path: Hello world!</title>
  </head>
  <body>
    <div style="width: 50%">
      <div style="width: 50%">Hello world!</div>
    </div>
  </body>
</html>
```



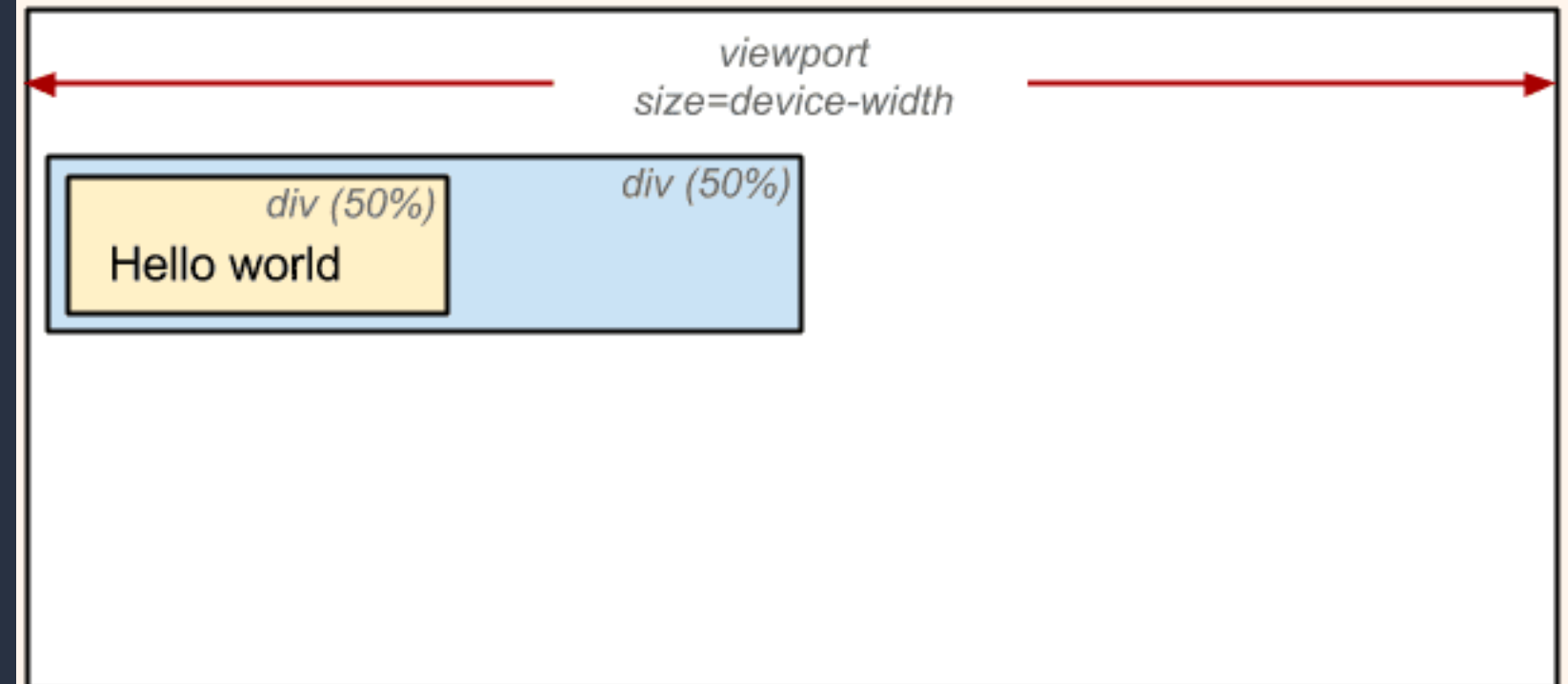
CRITICAL RENDERING PATH

1. Get the initial HTML
2. Parse the HTML into the DOM
3. Parse CSS into the CSSOM
4. Create render tree
5. Determine where render tree nodes should appear on the page (layout)
6. Paint nodes onto screen (rasterize)

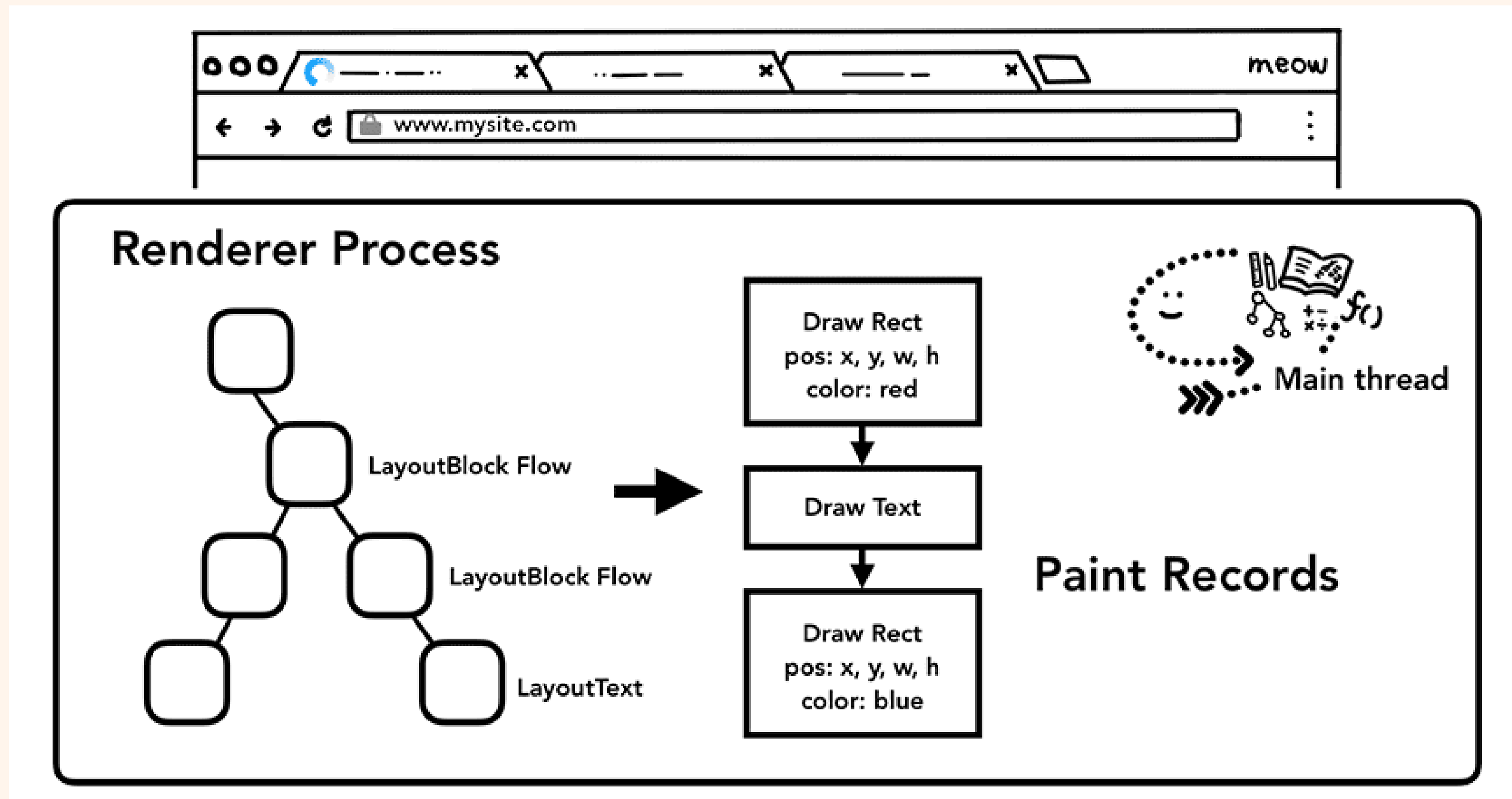


5. LAYOUT

```
<!DOCTYPE html>
<html>
  <head>
    <meta name="viewport" content="width=device-width,initial-scale=1" />
    <title>Critical Path: Hello world!</title>
  </head>
  <body>
    <div style="width: 50%">
      <div style="width: 50%">Hello world!</div>
    </div>
  </body>
</html>
```



6. RASTERIZE/PAINT



6. RASTERIZE/PAINT

Hello students



CS 303

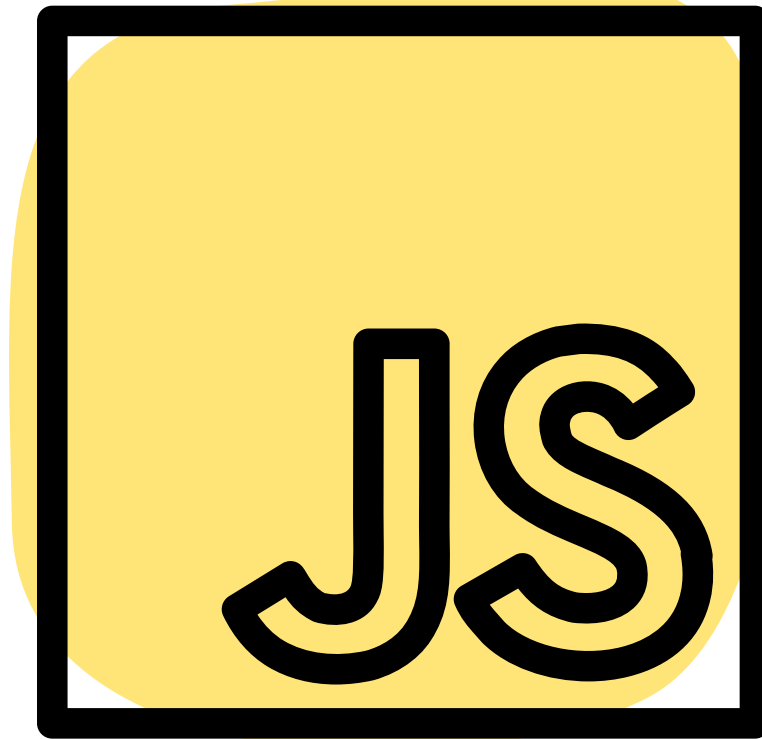
FRONTEND FRAMEWORKS

JAVASCRIPT IN THE FE

Purpose: to make web pages dynamic

Examples:

- Event listeners
- DOM manipulation
- Data fetching



EVENT LISTENERS

3 ways to add event listeners in vanilla js

```
<button id="button"
  onclick="console.log('clicked from html')">
  Click me!
</button>
```

1 way to remove

```
const button = document.querySelector("#button");

button.addEventListener("click", () => {
  console.log('add event listener click');
});
```

Types:

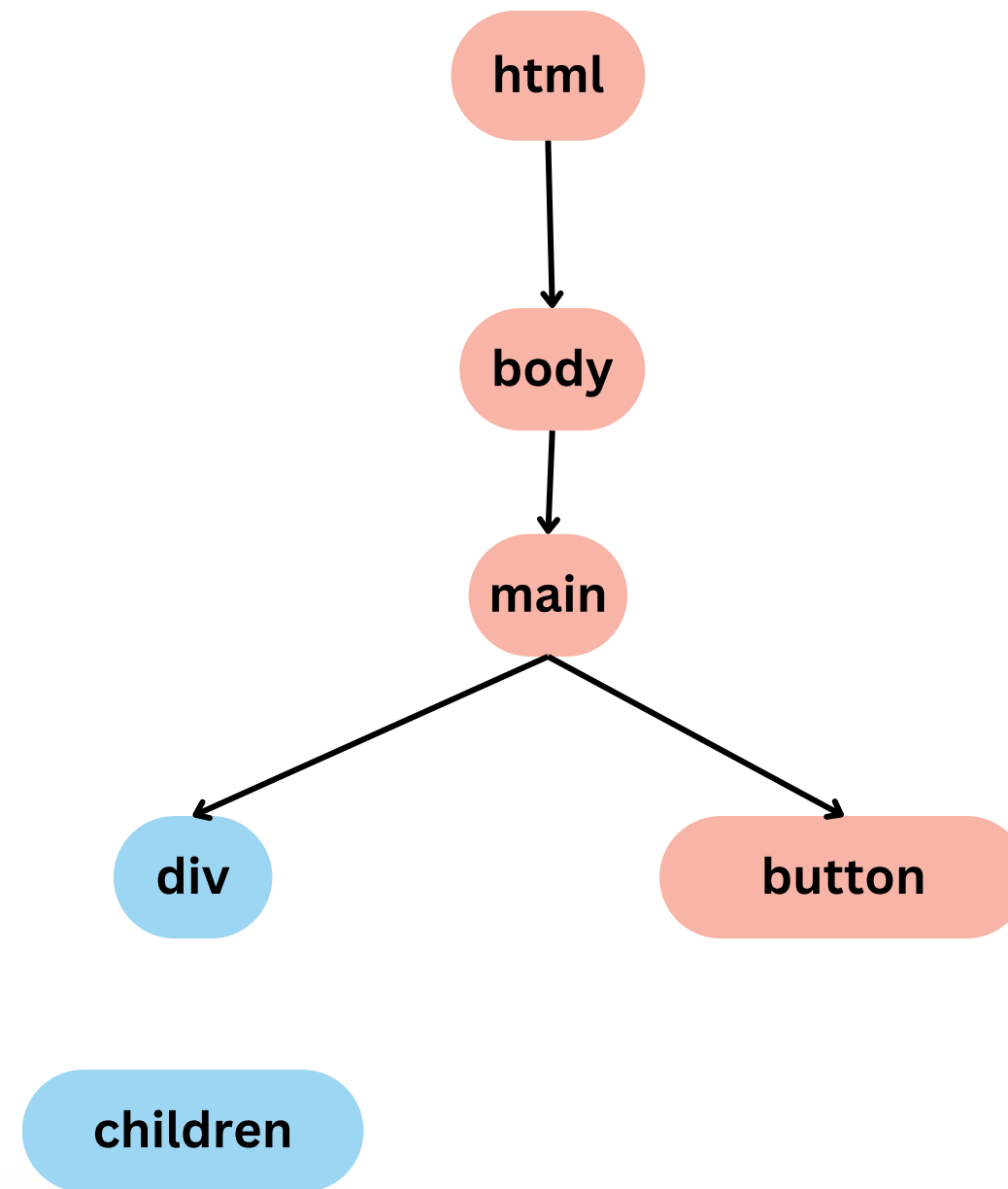
- click
- mouseover
- drag
- etc...

```
const button = document.querySelector("#button");
button.onclick = function() {
  console.log('clicked');
}
```

```
button.removeEventListener("click", buttonClickHandler);
```

EVENT PROPAGATION

Events will move up
the DOM tree

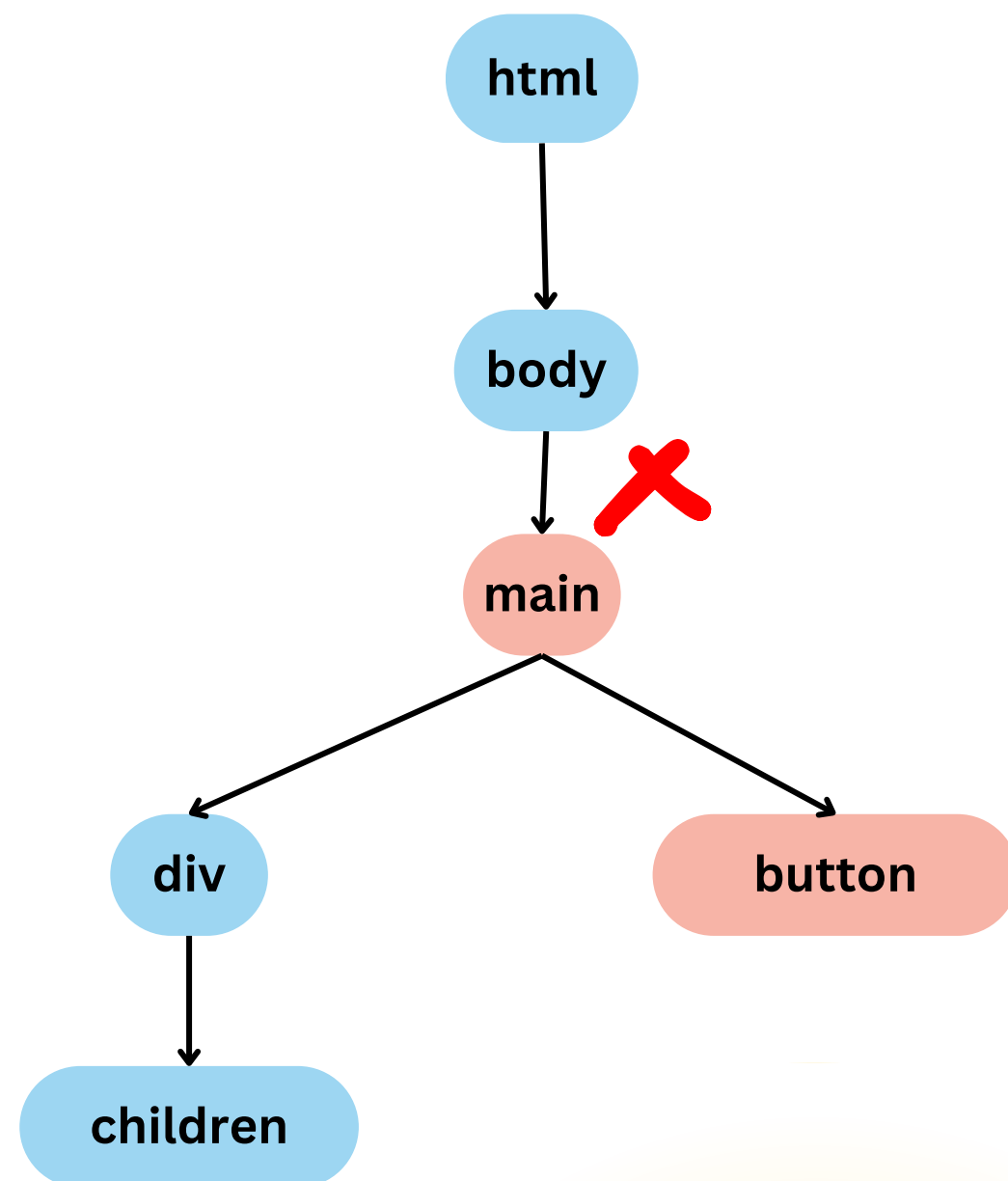


click fired

continues bubbling up the
dom tree to trigger other
handlers for click event

PREVENTING PROPAGATION

Use `preventDefault()` or `stopPropagation()`



```
main.addEventListener("click", (e) => {  
  e.stopPropagation();  
  console.log("main click handler");  
});
```

stackblitz.com/edit/cs303-js-todo

WHAT IS A FRAMEWORK?

Framework = scaffolding for building out a software system

- **Opinionated** – heavy lifting on how things should be rendered have been made
- **Declarative** – reduces boilerplate
- **Quicker ramp up** – expectations on how framework works is same regardless of which project you join
 - No bespoke solutions

4 COMMONALITIES

- 1.Components
- 2.State Handling
- 3.Special rendering (ex: VDOM)
- 4.Dependency Injection

4 COMMONALITIES

1. **Components**
2. State Handling
3. Special rendering (ex: VDOM)
4. Dependency Injection

COMPONENTS

What are components?

Reusable pieces of code that serve as UI building blocks

- Makes code maintainable

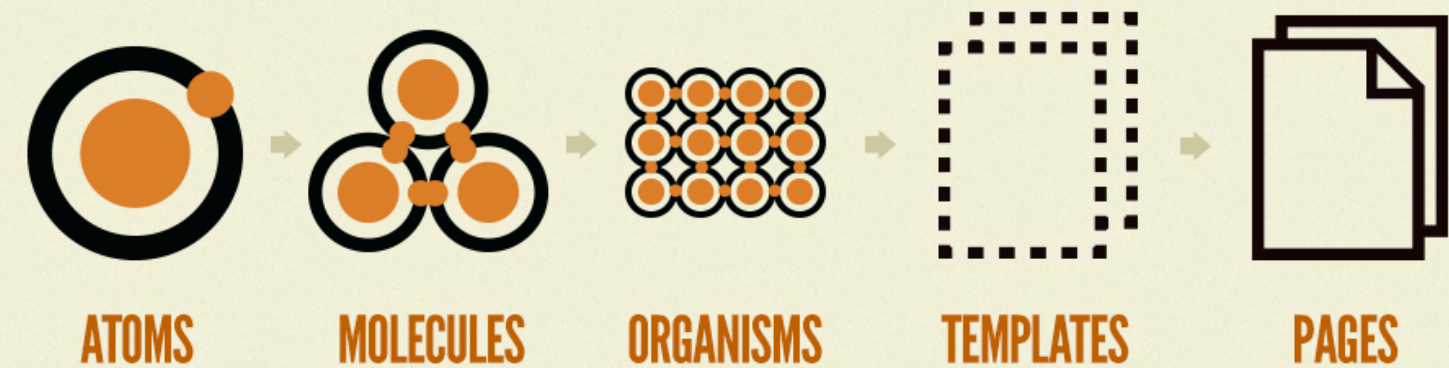
```
import React from 'react';

export default function MyComponent() {
  return <div>Hello, this is my React component!</div>;
}
```

COMPONENTS

Components Thinking:

- Breaking down components into the smallest usable unit and then building from here



COMPONENTS

What are components?

Reusable pieces of code that serve as UI building blocks

- Makes code maintainable

Components Thinking:

- Breaking down components into the smallest usable unit and build

```
import React from 'react';

export default function MyComponent() {
  return <div>Hello, this is my React component!</div>;
}
```

Your turn: With your partner, create a list of components from the to-do list

- Don't forget to capture this in your learning doc

stackblitz.com/edit/cs303-js-todo

4 COMMONALITIES

1. Components
2. **State Handling**
3. Special rendering (ex: VDOM)
4. Dependency Injection

STATE HANDLING IN THE FE

State = the user interface
based on the user interaction

Ex: toggling light/dark mode
on a web app

- **State:** mode <--- holds
'light' or 'dark'
- **Effect:** applies either light
or dark theme

```
export default function MyComponent() {  
  const [myState, setState] = useState('My state here ✨');  
  return (  
    <div>  
      Hello, this is my React component!  
      {myState}  
    </div>  
  );  
}
```

STATE HANDLING IN THE FE

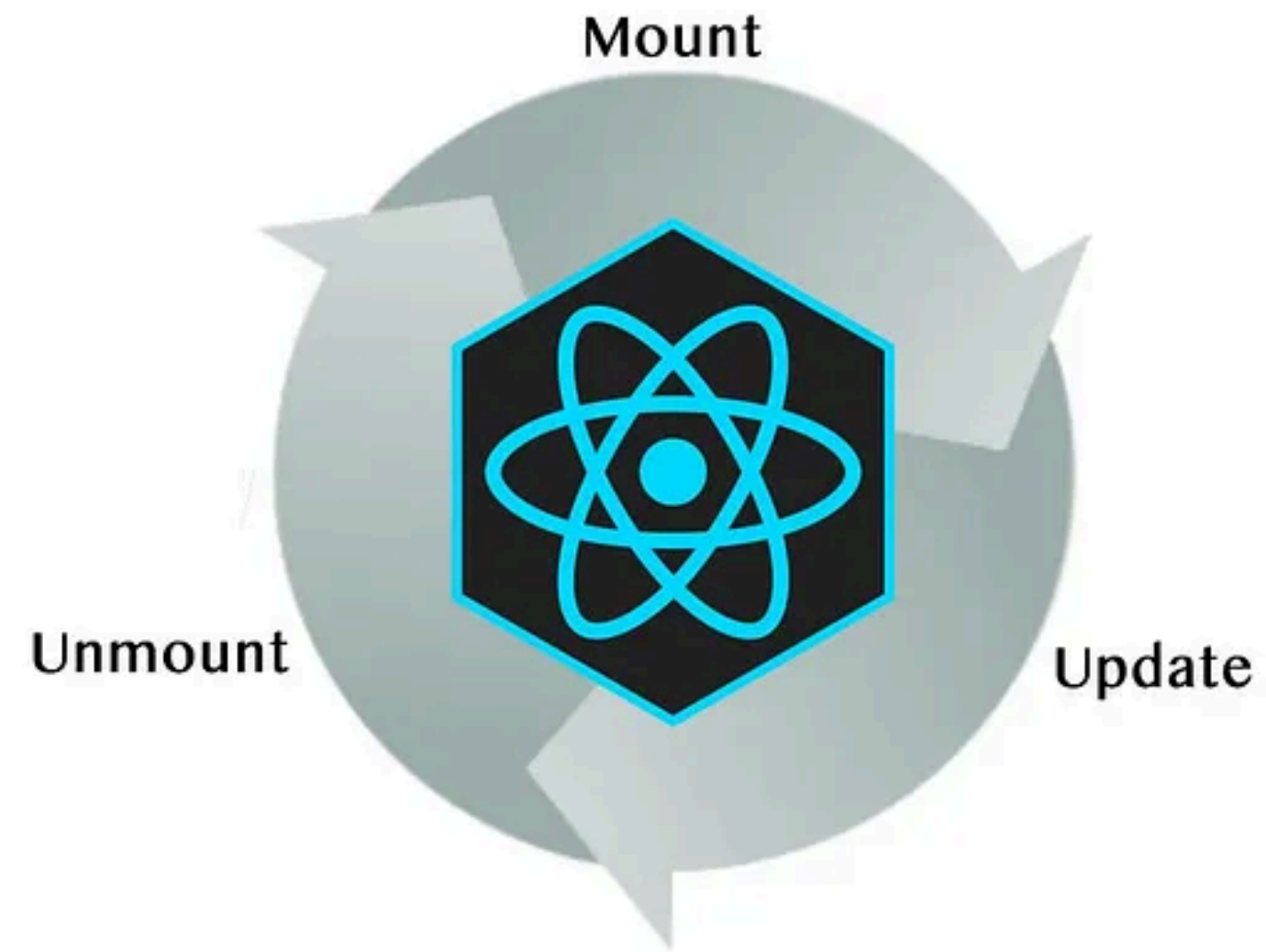
State = the user interface based on the user interaction

Your turn:

- With your partner – list the states that should be tracked for the todo item component
 - Capture this in your learning doc

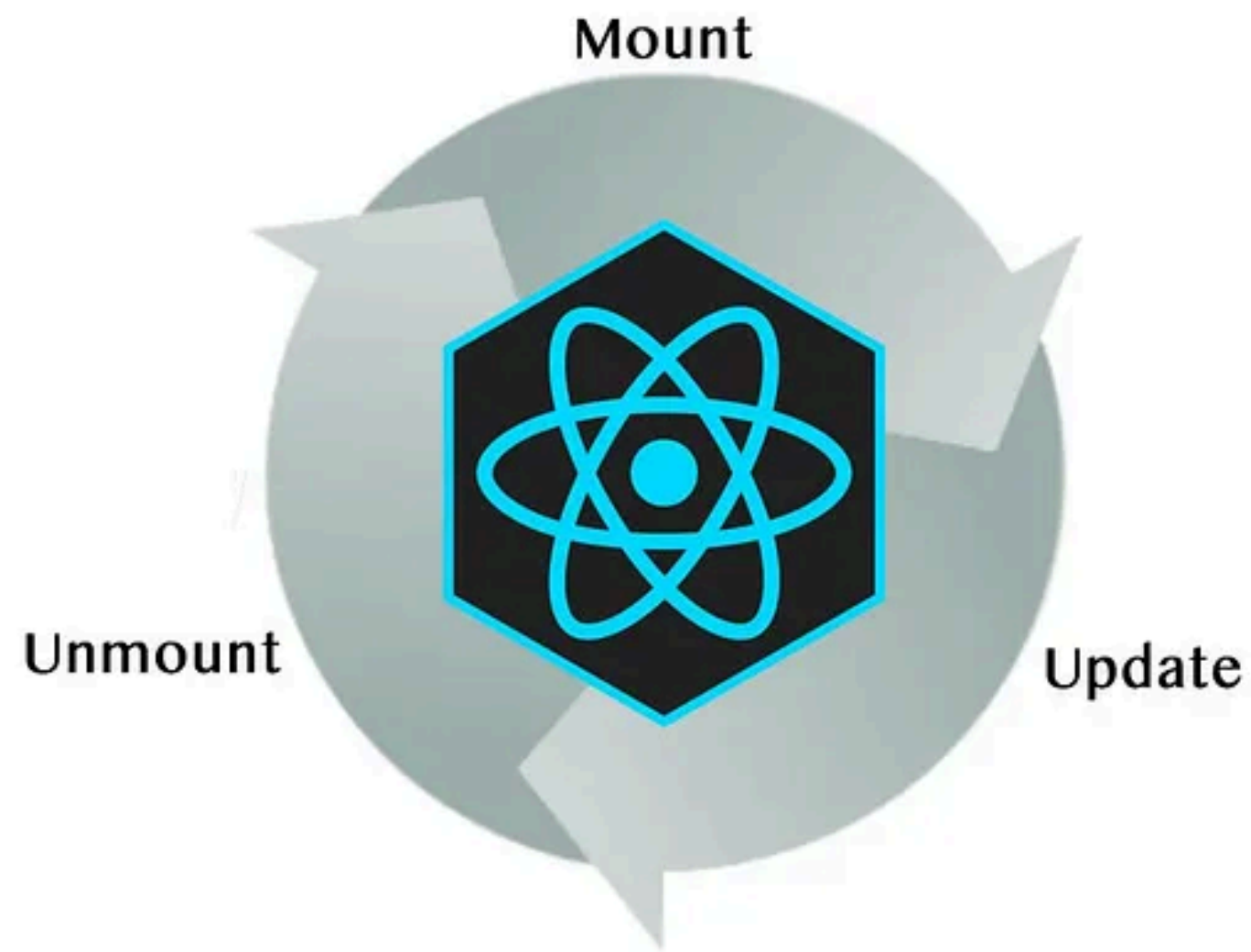
STATE HANDLING IN THE FE

Component Lifecycle



STATE HANDLING IN THE FE

Component Lifecycle



Trigger render

2 reasons to do this

1. Component's initial render
2. Component's state (or ancestor) has been updated

Render component

Commit to DOM

COMPONENTS + STATE

Put the 2 previous concepts together and translate the to-do list to React components + state

- Link to your forked stackblitz in your learning doc

stackblitz.com/edit/cs303-react-todos

PROJECT #1 OVERVIEW

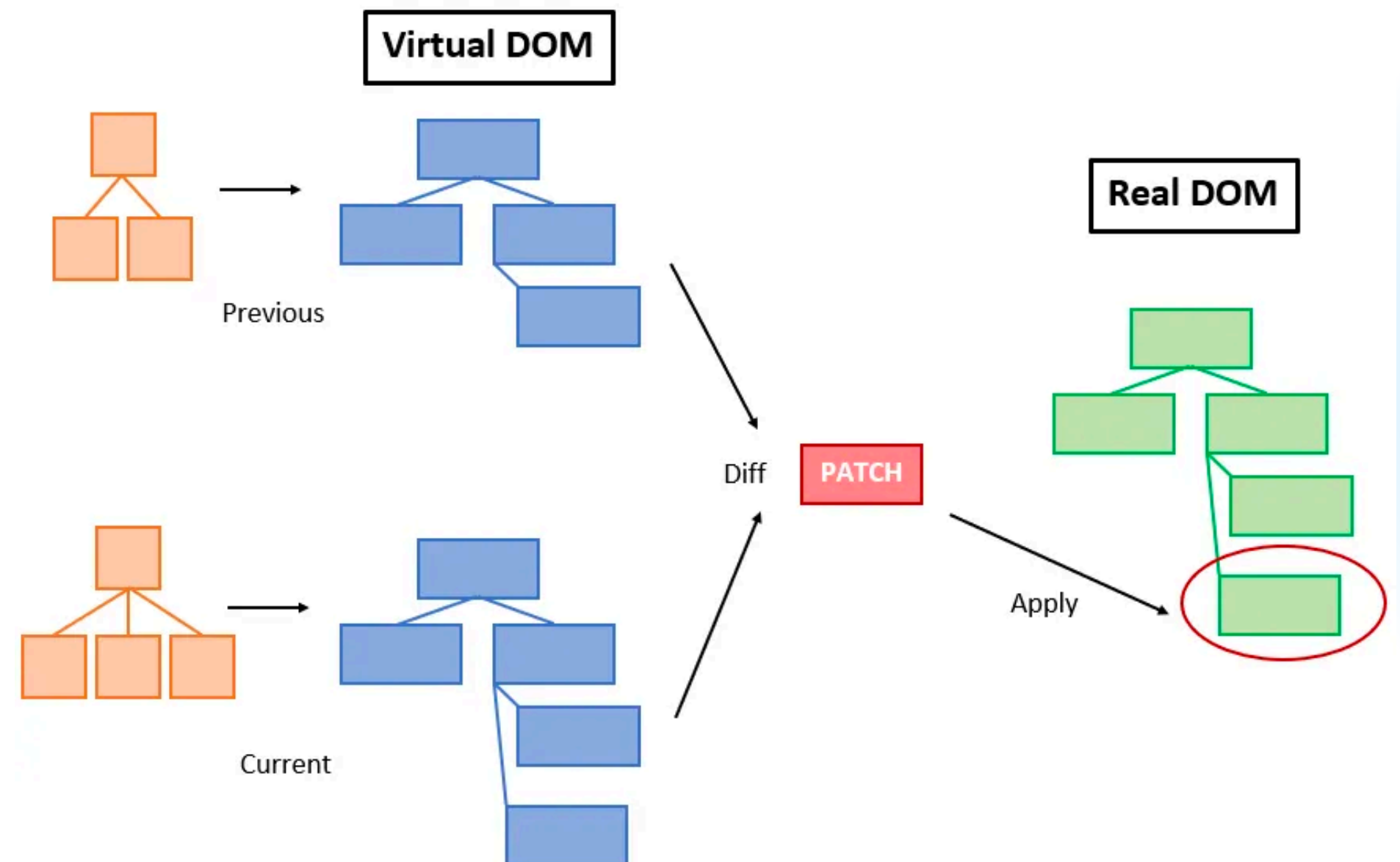
Project Spec

4 COMMONALITIES

1. Components
2. State Handling
3. **Special rendering (ex: VDOM)**
4. Dependency Injection

VIRTUAL DOM (VDOM)

- 1.Virtual DOM
- 2.When changes are made:
- 3.Rerender a new virtual DOM tree
- 4.Diff against previous virtual DOM tree
- 5.Make changes to the actual (physical) DOM



4 COMMONALITIES

1. Components
2. State Handling
3. Special rendering (ex: VDOM)
4. **Dependency Injection**